

EPHEC

Protection du système d'information d'une clinique médicale :

Création d'une entreprise fictive, analyse d'un point de vue « Protect » et mise en place de contre-mesures.

Élève :

Cours : Éléments de Protection de Cybersécurité

Professeur :

Année académique : 2025 - 2026

Section : Informatique orientation Sécurité des Systèmes

Table des matières

A. Contexte d'entreprise.....	3
1. Polyclinique Vandervelde (PVDV) – Membre du groupe Shirocks.....	3
2. Infrastructure du bâtiment.....	3
3. Infrastructure IT.....	3
4. Schéma de l'infrastructure IT.....	4
5. Actifs à protéger.....	5
6. Principales menaces et vecteurs d'attaque.....	6
7. Rôle de la fonction CyFun 2025 « <i>Protect</i> ».....	8
7.1. Pour la Polyclinique Vandervelde.....	8
7.2. Par rapport aux autres fonctions CyFun.....	8
7.3. Résultats visés par la fonction <i>Protect</i>	9
7.3.1. Contrôle d'accès – PR.AA :.....	9
7.3.2. Sensibilisation et formation – PR.AT :.....	9
7.3.3. Sécurité des données – PR.DS :.....	9
7.3.4. Sécurité des plateformes – PR.PS :.....	10
7.3.5. Résilience de infrastructures – PR.IR :.....	10
B. Niveau d'assurance.....	11
1. Évaluation du niveau d'assurance requis.....	11
1.1. PVDV considérée comme « <i>Small</i> ».....	12
1.2. PVDV considérée comme « <i>Large</i> ».....	13
2. Choix retenu.....	14
3. Mise en œuvre.....	14
C. Contre-mesures.....	15
1. Prémices.....	15
2. Étape 1 : introduction physique dans un local « sécurisé ».....	15
2.1. Contexte et contre-mesures.....	15
2.2. Configuration initiale.....	15
2.3. Démonstration d'une menace potentielle.....	16
2.4. Démonstration de contre-mesures.....	16
2.5. Limite de ces contre-mesures.....	17
2.6. Approche pour l'institutionnalisation.....	17
2.6.1. Politique.....	17
2.6.2. Procédures.....	17
3. Étape 2 : une attaque de type « <i>BadUSB</i> ».....	18
3.1. Contexte et contre-mesures.....	18
3.2. Configuration.....	18
3.2.1. Résumé des étapes de configuration.....	19
3.2.2. Installation de USBGuard.....	20
3.2.3. Installation de Wazuh.....	22
3.3. Démonstration d'une menace potentielle.....	24
3.4. Démonstration du fonctionnement de la contre-mesure.....	25
3.5. Limites de la contre-mesure.....	26
3.6. Approche pour l'institutionnalisation de la contre-mesure.....	27
3.6.1. Politique interne.....	28
3.6.2. Procédures (focus sur les port USB).....	28
4. Étape 3 : le rançongiciel.....	29
4.1. Contexte et contre-mesure.....	29
4.2. Configuration.....	29

4.2.1. Résumé des étapes de configuration.....	30
4.3. Démonstration d'une menace potentielle.....	33
4.4. Démonstration du fonctionnement de la contre-mesure.....	34
4.5. Limites de la contre-mesure.....	43
4.6. Approche pour l'institutionnalisation de la contre-mesure.....	44
4.6.1. Politique.....	44
4.6.2. Procédures.....	45
D. Conclusions.....	46

A. Contexte d'entreprise

1. Polyclinique Vandervelde (PVDV) – Membre du groupe Shirocks

La polyclinique Vandervelde est un centre de soins médicaux de divers types (médecine généraliste, dermatologie, orthopédie, etc.). On y pratique uniquement des consultations, examens et petites chirurgies, aucune hospitalisation n'y est faite. Située à Bruxelles en Belgique, elle fait partie du grand groupe de centres hospitaliers « Shirocks ».

2. Infrastructure du bâtiment

PVDV est composée d'une salle d'accueil (occupée par des secrétaires médicaux), 10 salles de soins pouvant être occupées par des médecins, 4 salles d'examens (type radiographie, prise de sang, etc.) et diverses salles/armoires où sont entreposées du matériel médical. Une des salles, où sont stockés des médicaments nécessitant un certain niveau de protection physique, accueille une armoire à rack serveurs pour les divers usages locaux d'informatique de la polyclinique.

3. Infrastructure IT

Du fait que PVDV fasse partie du groupe Shirocks, la polyclinique délocalise géographiquement une grande partie de ses services IT. Les secrétaires médicaux, infirmier·ère·s et médecins doivent, d'une manière ou d'une autre, avoir accès au système informatique local de PVDV.

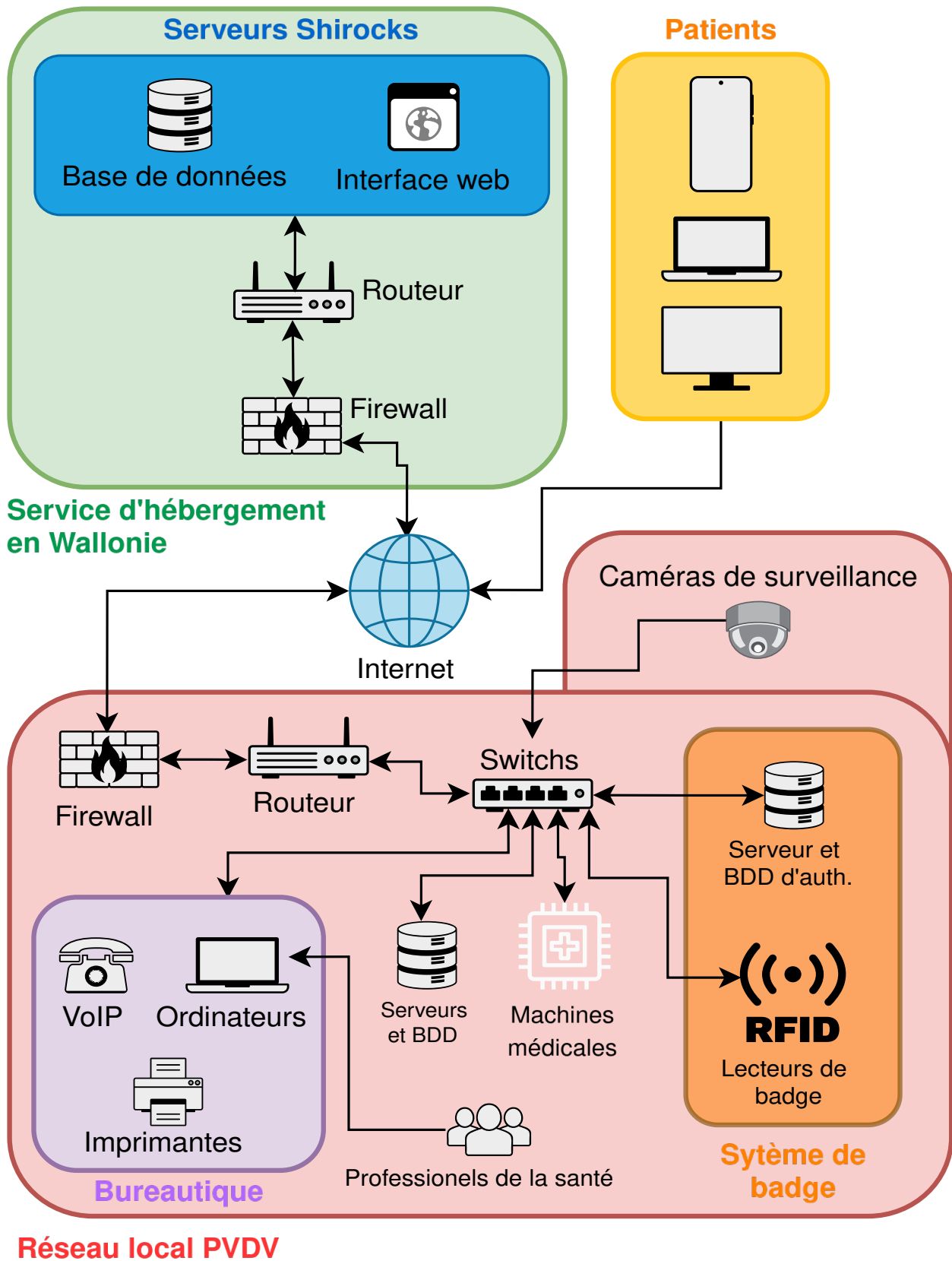
Tout le personnel médical de PVDV accède aux informations médicales des patients (consultations, examens, documents divers, informations personnelles, etc.) à travers la plateforme centralisée de Shirocks. Le groupe loue des serveurs en Wallonie pour l'hébergement de ses deux interfaces web (patients et professionnel de la santé) et le stockage de données. Ils louent également un duplicata de ces serveurs (web et données) dans deux autres pays distincts.

PVDV dispose malgré tout d'une petite base de données locale qui sert à stocker des informations propres à la polyclinique, des copies locales des données médicales (faites avant d'être envoyées au serveur Shirock), le site web de la clinique ainsi que des *backups* des machines locales et de leurs logs.

Les machines médicales de la clinique qui ont besoin d'une connexion au réseau sont connectées à celui-ci via divers commutateurs.

Enfin, divers commutateurs sont branchés à travers la polyclinique et mènent vers un commutateur central connecté à un routeur. Sur ces *switchs* sont branchés les ordinateurs, téléphones VoIP, imprimantes et machines médicales, ainsi que le système d'authentification (comportant un petit serveur avec BDD et les multiples lecteurs de badges) et le système de caméras de surveillance.

4. Schéma de l'infrastructure IT



5. Actifs à protéger

Les actifs à protéger par PVDV sont tant géographiquement locaux que délocalisés, et tant matériels qu'immatériels.

- **Actifs matériels :**

- Tout le matériel onéreux de la clinique.
- Les médicaments (particulièrement ceux soumis à prescription).
- Tout l'équipement connecté au réseau local (routeurs, *switchs*, racks serveurs, lecteurs de badges, caméras, etc.).
- Les documents papier contenant des informations sensibles et/ou protégées par le RGPD.

- **Actifs immatériels :**

- Les données privées et RGPD circulant sur le réseau local et parfois jusqu'au serveurs Shirocks distants.
- Les données de paiements (standard PCI-DSS).
- Les systèmes d'exploitations et logiciels tournant sur **toutes** les machines locales (serveurs, ordinateurs, téléphones VoIP, machines médicales, caméras de surveillance, *switchs*, routeurs, serveurs et imprimantes) :
 - Les droits d'accès.
 - L'authentification.
 - Les données (« en transit », « au repos », « en cours d'utilisation », logs, *backups*).
- Les serveurs distants de Shirocks (dans une certaine mesure).

6. Principales menaces et vecteurs d'attaque

Les centres médicaux sont souvent des cibles de choix lors de cyberattaques (sans oublier les risques environnementaux pouvant avoir un impact sur la cybersécurité mais que je ne traitera pas dans ce rapport). Les motivations des acteurs de la menace peuvent être variées. Selon un [rapport](#) de l'agence européenne ENISA, voici les principales menaces pour le secteur de la santé, les types d'acteurs de la menace et leurs motivations :

Figure 5: Threats in the health sector (January 2021 to March 2023)

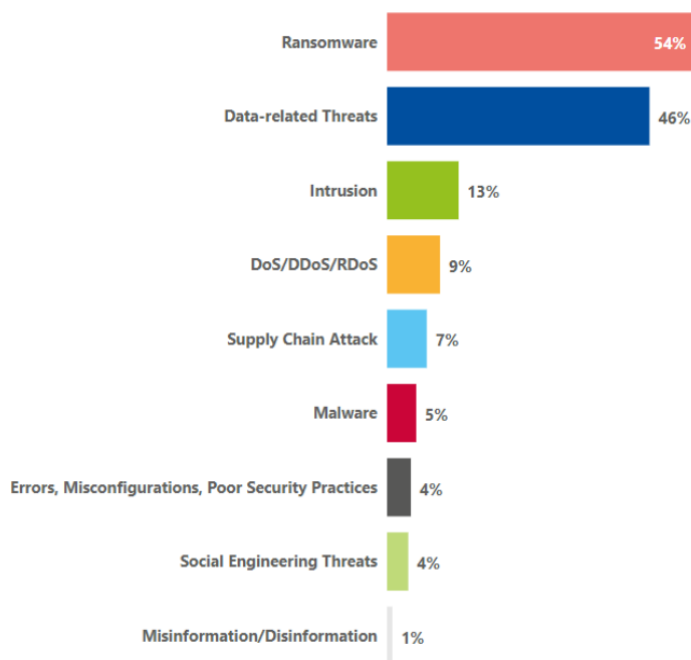


Figure 7: Actor types

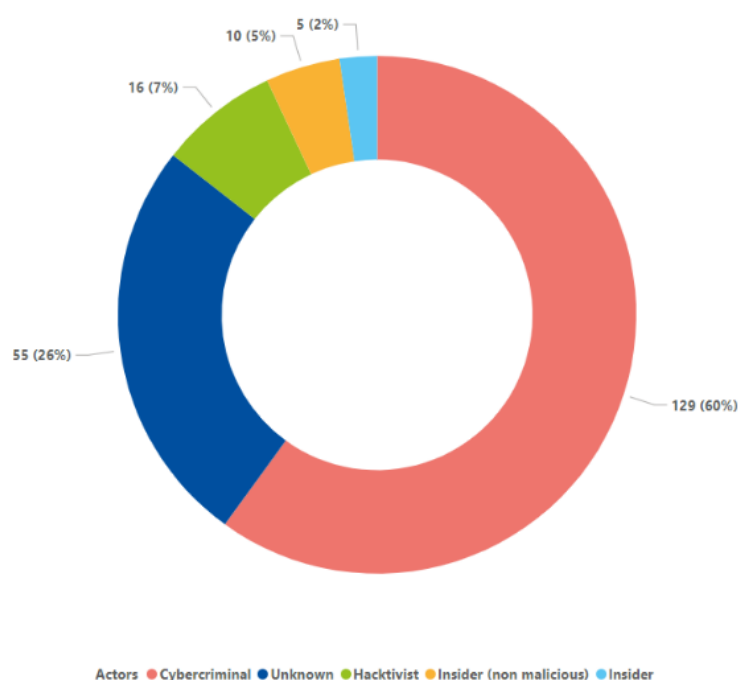


Figure 8: Motivation

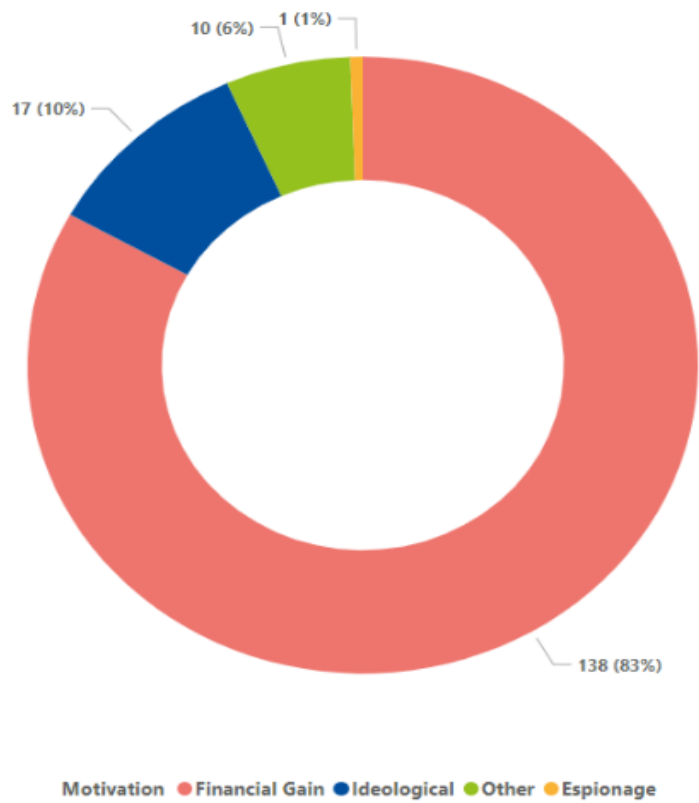
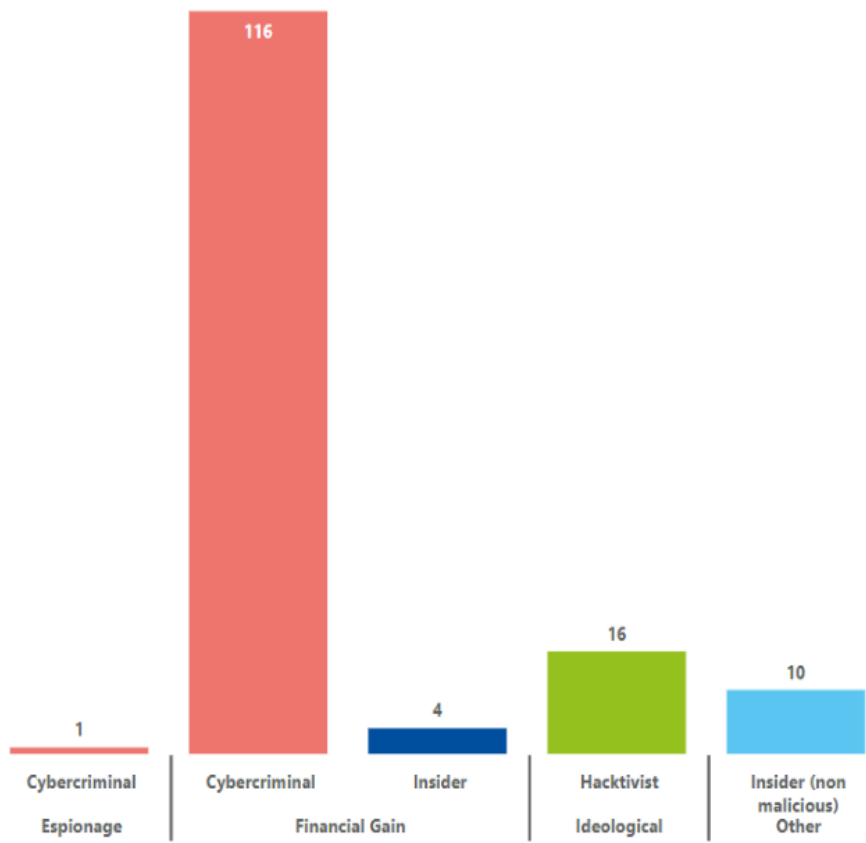


Figure 9: Motivation and actors



7. Rôle de la fonction CyFun 2025 « *Protect* »

7.1. Pour la Polyclinique Vandervelde

Dans le contexte de PVDV, le rôle fondamental de la fonction *Protect* est de mettre en œuvre des contrôles et des barrières de sécurité pour garantir la prestation des services médicaux critiques et limiter l'impact financier ou réputationnel d'un événement de cybersécurité potentiel.

Puisque PVDV est une structure hybride (infrastructure locale mais dépendance forte au groupe Shirocks pour les données patients), la fonction de protection doit agir sur deux fronts :

- **Sécurisation du périmètre local** : Protéger les actifs physiques et numériques présents dans le bâtiment bruxellois. Cela inclut la sécurisation de l'armoire serveur située dans la salle de stockage des médicaments, le contrôle d'accès via les lecteurs de badges, et la sécurisation du réseau local sur lequel sont connectés les machines médicales et les postes de travail.
- **Sécurisation de l'interface vers l'extérieur** : puisque les données médicales transitent vers les serveurs du groupe Shirocks et que des *backups* locaux sont effectués, la fonction *Protect* doit assurer que ces canaux de communication sont chiffrés et que l'authentification des utilisateur·rice·s (médecins, infirmier·ère·s, secrétaires) est robuste pour éviter les intrusions. Elle doit aussi s'assurer que ces utilisateur·rice·s soient formés et sensibilisés aux risques liés à la cybersécurité.

7.2. Par rapport aux autres fonctions CyFun

La fonction *Protect* n'est pas suffisante à elle seule. Elle interagit avec les autres fonctions du *framework* :

- **Par rapport à *Identify*** : La protection s'appuie directement sur l'inventaire réalisé dans la phase *Identify*. On ne peut pas protéger ce qu'on ne connaît pas. PVDV a identifié ses actifs critiques (médicaments, racks serveurs, données RGPD, matériel médical). La fonction *Protect* est l'application concrète de mesures de sécurité sur ces actifs.
- **Par rapport à *Detect*** : La protection fournit les règles et les systèmes que la fonction *Detect* va surveiller. Par exemple, PVDV a installé un *firewall* et un système de badges. La fonction *Protect* configure ces outils pour bloquer les accès non autorisés. Ensuite, la fonction *Detect* analysera les logs générés par ces outils pour repérer si une tentative de contournement de ces protections a eu lieu.
- **Par rapport à *Respond* et *Recover*** : Le but de la protection est de rendre les fonctions *Respond* et *Recover* moins nécessaires ou moins difficiles à exécuter. Une bonne protection – comme la mise en place de *backups* réguliers des machines locales – facilite grandement la récupération en cas de succès d'une attaque par *Ransomware*, limitant ainsi le besoin d'enclencher un plan de réponse.

7.3. Résultats visés par la fonction *Protect*

Selon le [document Cyfun 2025](#) pour le secteur essentiel, la mise en œuvre de la fonction de protection doit aboutir aux résultats suivant :

7.3.1. Contrôle d'accès – PR.AA :

« Access to physical and logical assets is limited to authorised users, services, and hardware and managed commensurate with the assessed risk of unauthorised access ».

Objectif PVDV :

L'accès aux actifs matériels et logiciels cités précédemment doit être réfléchi et implémenté avec les logiques *Zero-Trust* et *Least Privileged*. Permettant ainsi de mitiger les risques liés à des accès non autorisés. Ces risques doivent être méticuleusement estimés pour rendre les mesures de protection proportionnées au risque et donc efficaces en termes de coût. Des politiques doivent être mises en place, tenues à jour et respectées par le personnel.

7.3.2. Sensibilisation et formation – PR.AT :

« The organisation's personnel are provided with cybersecurity awareness and training, so that they can perform their cybersecurity-related tasks ».

Objectif PVDV :

Tout le personnel médical du centre doit obligatoirement participer à des séances de sensibilisation et formation concernant la cybersécurité. Ces séances auront un contenu adapté au personnel qui les suit. Elles devront avoir lieu au moins deux fois par an pour s'assurer que des nouveaux membres du personnel rejoignant la clinique soient formés au plus vite. Seront privilégiées les séances en présentiel et en groupe, avec des exercices participatifs, engageants et ludiques.

7.3.3. Sécurité des données – PR.DS :

« Data are managed consistent with the organisation's risk strategy to protect the confidentiality, integrity, and availability of information ».

Objectif PVDV :

Les données médicales manipulées quotidiennement au sein de PVDV sont soumises au RGPD. De fait, des mesures appropriées doivent être appliquées et des mesures supplémentaires – propres aux risques élevés d'attaques cyber dans le secteur médical et à la nature sensible des activités – doivent être pensées et assurées pour pallier au risque d'incident grave.

Concrètement, un des objectifs serait d'avoir un système de secours permettant de maintenir la disponibilité des données même en cas d'attaque (pour pouvoir continuer de soigner les patients efficacement).

7.3.4. Sécurité des plateformes – PR.PS :

« The hardware, software (e.g., firmware, operating systems, applications), and services of physical and virtual platforms are managed consistent with the organisation's risk strategy to protect their confidentiality, integrity, and availability ».

Objectif PVDV :

Pour garantir la sécurité des routeurs, *switchs*, et autres matériel de la clinique, PVDV doit utiliser des outils de configuration comme Ansible pour avoir une configuration reproductible facilement à l'identique et de façon sécurisée. Le personnel adéquat devra aussi veiller à la mise à jour régulière de ces « plateformes » (potentiellement via certains outils d'automatisation). La durée de vie du matériel devra être prise en compte pour éviter de se retrouver avec du matériel faillible dû à son âge. Dans le cas de machines médicales trop coûteuses à remplacer, il faudra alors ajouter à celles-ci une couche de protection supplémentaire.

7.3.5. Résilience de infrastructures – PR.IR :

« Security architectures are managed with the organisation's risk strategy to protect asset confidentiality, integrity, and availability, and organisational resilience ».

Objectif PVDV :

Pour protéger l'infrastructure entière de PVDV, il est impératif de réfléchir à toutes les interactions possibles entre ses divers éléments. Afin de limiter les dégâts et maintenir les diverses activités médicales en cas d'attaque on pourrait viser des objectifs tels que la segmentation / micro-segmentation des divers tronçons de l'infrastructure et la mise en place d'un système de secours qui prendrait le relais en cas de panne généralisée. La résilience de certaines parties de l'infrastructure d'un point de vue macro dépendent de Shirocks. Par exemple, dans le cas où l'interface web permettant au personnel de PVDV d'accéder à un résultat de radiographie venait à subir une attaque par déni de service, Shirocks se doit d'utiliser un outil qui absorberait l'attaque (Cloudflare, etc.).

B. Niveau d'assurance

1. Évaluation du niveau d'assurance requis

Afin de définir le niveau d'assurance requis pour PVDV, j'ai utilisé un outil développé par le CSIRT (*Computer Security Incident Response Team*) belge, le CCB (*Center for Cybersecurity Belgium*). Cet outil est une simple feuille de calcul préconfigurée dans laquelle il suffit de changer des paramètres pour afficher un score correspondant à l'un des trois niveaux d'assurance (*BASIC, IMPORTANT, ESSENTIAL*). Grâce à [ce site web](#), j'ai pu suivre un guide me permettant de changer ces paramètres de manière pertinente.

Dès le premier paramètre à changer j'ai rencontré un « problème », de quelle taille dois-je considérer PVDV (*Small, Medium, Large*) ?

Size Number	Size	Size criteria *
3	Large	250 or more employees**, OR more than EUR 50 million annual turnover, OR more than EUR 43 million annual balance sheet total.
2	Medium	In between Small and Large
1	Small	Less than 50 employees AND less than EUR 10 million annual turnover AND less than EUR 10 million annual balance sheet total

La réponse simple est « *Small* » car PVDV emploie moins de 50 employés et ne génère pas un si grand chiffre d'affaire. Mais si je considère PVDV comme membre de l'organisation Shirocks, la clinique doit être élevée au rang de « *Large* ». Cela complique l'évaluation. Ainsi, pour assurer une analyse complète, j'ai réalisé deux évaluations. En comparant leurs résultats, je pourrai décider de façon justifiée quel niveau me semble le plus approprié.

1.1. PVDV considérée comme « Small »



CENTRE FOR
CYBERSECURITY
BELGIUM

Version: 2024-01-08

Healthcare			Common skills		Common skills		Common skills		Extended Skills		Extended Skills			
Organization Size (L/M/S = 3/2/1)	1	Threat Actor Type	Competitors		Ideologues Hactivists		Terrorist		Cyber Criminals		Nation State actor			
Cyber Attack Category	Global or Targetted	Impact	Prob	Risk Score	Prob	Risk Score	Prob	Risk Score	Prob	Risk Score	Prob	Risk Score		
Sabotage/ Disruption (DDOS,...)	2	Med	Low	0	Med	5	Med	5	Low	0	Med	5		
Information Theft (espionage, ...)	2	Med	Low	0	Med	5	Med	5	Med	5	Med	5		
Crime (Ransom attacks)	1	Med	Low	0	Low	0	Low	0	High	5	Med	2,5		
Hactivism (Subversion, defacement...)	1	Low	Low	0	Low	0	Med	0	Low	0	Low	0		
Disinformation (political influencing)	1	Med	Low	0	Med	2,5	Low	0	Low	0	Med	2,5	Score	CyFun Level
Total	Total			0		12,5		10		10		15	47,5	BASIC

Pour cette évaluation, PVDV est considérée comme « Small ». J'ai également changé l'impact que pouvait avoir certaines catégories de cyberattaques (dans l'encadré mauve) en me référant à [ce guide](#). En effet, ces trois catégories sont préconfigurées avec le niveau d'impact « High ». Or ce dernier implique que pour un incident ayant lieu dans le secteur de la santé 200 personnes meurent, soient atteintes de maladies chroniques ou d'handicaps à long terme. Si on considère l'impact comme ayant effet uniquement sur la clinique, il est impossible d'arriver à ce stade. J'ai donc réduit le niveau d'impact à « Medium ». J'ai également changé d'autres paramètres (encadrés en rose) concernant la probabilité que certaines attaques soient perpétrées par certains acteurs de la menace. En effet, en lisant le [rapport](#) de l'ENISA précédemment cité et en prenant en compte des tensions actuelles dans le climat géo-politique (guerre en Ukraine, pressions de *lobbying* des États-Unis sur l'Europe, etc.), je me suis permis de revoir ces probabilités à la hausse. Néanmoins, dû à l'impact réduit à « Medium » et la petite taille de PVDV, le score reste très bas. Considérant ainsi la clinique comme organisation ayant besoin d'un niveau d'assurance « BASIC ».

1.2. PVDV considérée comme « Large »



CENTRE FOR
CYBERSECURITY
BELGIUM

Version: 2024-01-08

Healthcare															
				Common skills		Common skills		Common skills		Extended Skills		Extended Skills			
Organization Size (L/M/S = 3/2/1)		3	Threat Actor Type	Competitors		Ideologues Hactivists		Terrorist		Cyber Criminals		Nation State actor			
Cyber Attack Category		Global or Targetted	Impact	Prob	Risk Score	Prob	Risk Score	Prob	Risk Score	Prob	Risk Score	Prob	Risk Score		
Sabotage/ Disruption (DDOS,...)		2	High	Low	0	Med	30	Med	30	Low	0	High	60		
Information Theft (espionage, ...)		2	High	Low	0	Med	30	Med	30	Med	30	Med	30		
Crime (Ransom attacks)		1	High	Low	0	Low	0	Low	0	High	30	Med	15		
Hactivism (Subversion, defacement...)		1	Low	Low	0	Low	0	Med	0	Low	0	Low	0		
Disinformation (political influencing)		1	Med	Low	0	Med	7,5	Low	0	Low	0	Med	7,5	Score	CyFun Level
Total		Total	Total		0		67,5		60		60		112,5	300	ESSENTIAL

Pour cette évaluation, PVDV est considérée comme « Large ». La logique derrière cette considération me vient d'un cours de l'année précédente où nous avons étudié ISO27001 et la méthode de gestion du risque EBIOS. On nous y explique que les fournisseurs doivent être considérés comme le premier vecteur d'attaque. Par fournisseurs on entend aussi partenaires, affiliés, franchisés et succursales. De fait, PVDV s'inscrit en tant que membre de l'écosystème Shirocks. De part sa position de petite clinique, avec un budget et des moyens limités, elle est même une cible de choix pour les acteurs de la menace ayant comme cible finale le groupe Shirocks (et ses gros hôpitaux). À nouveau, pour mieux refléter le climat actuel (le rapport de l'ENISA n'analysant que la période de janvier 2021 à mars 2023), j'ai changé certains paramètres (encadrés en rose). Même sans avoir changé ceux-ci, le simple fait d'être une organisation « Large » et d'avoir des impacts « High » augmente déjà drastiquement le score. Ainsi, la clinique est considéré comme une organisation ayant besoin d'un niveau d'assurance « ESSENTIAL ».

2. Choix retenu

Dans un [document](#) du service officiel belge de « Santé publique / Sécurité de la Chaîne alimentaire / Environnement », j'ai pu réaliser un calcul simple. Je ne prétends pas aux compétences d'un économiste, il y aura donc sûrement des facteurs que j'oublie de prendre en compte.

Pour la période du 1^{er} juillet 2024 au 1^{er} juillet 2025, le budget dédié à la cybersécurité dans les hôpitaux (publics et privés) en Belgique s'élève à 5.548.502,50 €. Pour cette même période, le budget total (soins, matériel médical, informatique, etc.) s'élève à 12.722.402.069,16 €. Les dépenses en cybersécurité représentent donc **0,044 %** du budget total. Bien qu'il y ai certainement d'autres paramètres à prendre en compte, je pense que ça reflète assez bien l'aspect minoritaire qu'occupe la cybersécurité au sein des hôpitaux.

Ainsi, pour une clinique de petite taille comme PVDV, il est difficile d'imaginer un système d'information qui soit protégé au plus haut niveau des standards NIS2. Néanmoins, dû à sa position « délicate », à la nature des données qu'elle traite et aux impacts potentiels sur la santé, PVDV se doit de chercher à atteindre les requis pour le niveau *ESSENTIAL* dans la limite de ce que permet le budget qui lui est allouée.

C'est pour cette raison que, selon moi, la manière la plus réaliste de procéder est la suivante : PVDV doit commencer par remplir les quotas lui permettant d'atteindre le niveau *SMALL*, ensuite, elle pourra progressivement entamer son ascension vers le niveau *ESSENTIAL*. La quantité de travail sera divisée sur les années à venir rendant ainsi la charge de travail et la charge mentale amoindrie pour les diverses personnes qui devront travailler à cette mise en conformité.

3. Mise en œuvre

Pour des raisons de praticité, je choisis de baser l'implémentation de contre-mesures *Protect* sur la [version 2025 de CyFun pour les organisations considérées comme IMPORTANT](#). Comme mentionné précédemment, PVDV est dans un entre-deux, donc je vais partir du principe que, pour cet exercice, la clinique a déjà fait un effort afin de répondre aux demandes du niveau *SMALL*.

C. Contre-mesures

1. Prémices

J'ai choisi d'articuler mes contre-mesures autour d'un scénario inventé par mes soins et inspiré de faits réels :

Un groupe d'activistes anti-vaccins cherche à saboter l'infrastructure informatique de petites et moyennes cliniques belges dans le but d'empêcher celles-ci d'avoir accès à leurs agendas digitaux de rendez-vous pour la vaccination au COVID-26. Parmi ces activistes se trouvent aussi des *hacktivistes*. Ces derniers ont eu la « bonne » idée de mettre en place un plan qui servira les intérêts de leur cause, fera croire à une attaque par un groupe de *hacker* sévissant actuellement dans le domaine des rançongiciels et leur permettra même d'extorquer une quantité considérable d'argent aux victimes.

PVDV est au courant de son retard en matière de cybersécurité. Heureusement, le groupe Shirocks, avec qui PVDV partage certaines responsabilités juridique, a missionné une équipe d'ingénieurs en cybersécurité (avec des profils tant techniques que gouvernance) de trouver des solutions réalistes en terme de budget pour sécuriser les systèmes informatiques de ses « maillons faible » comme PVDV.

2. Étape 1 : introduction physique dans un local « sécurisé »

2.1. Contexte et contre-mesures

Bien que PVDV aie investi dans un système de badge d'accès pour les portes nécessitant un certain niveau de protection, il existe encore quelques points faillibles à la sécurité physique du lieu. L'équipe d'ingénieur, en collaboration avec d'autres personnes spécialisées en sécurité physique, doivent passer en revue la clinique et définir les points d'amélioration.

2.2. Configuration initiale

Pour cette étape 1, il n'y a pas de configuration à faire mais plutôt des constats :

Toutes les portes de PVDV ne sont pas très modernes, le bâtiment est vieux et rénové par ci et là. Les poignées avec lecteur de badge intégré qui ont été installées sur les plus vieille porte sont parfois inefficaces. Les portes plus modernes donnent l'air d'être sécurisées mais ont parfois des vulnérabilités critiques qui pourraient être contrecarrées aisément.

Le système de badge est suffisamment moderne pour empêcher les novices de s'y attaquer sans badge d'accès, mais la technologie des badges d'accès laisse à désirer. Nous sommes informés par le personnel de PVDV qu'il n'a jamais reçu de formation de sensibilisation à la sécurité physique.

Les clés pour ouvrir armoires, tiroirs et autres ne sont pas stockées de manière sécurisée.

2.3. Démonstration d'une menace potentielle

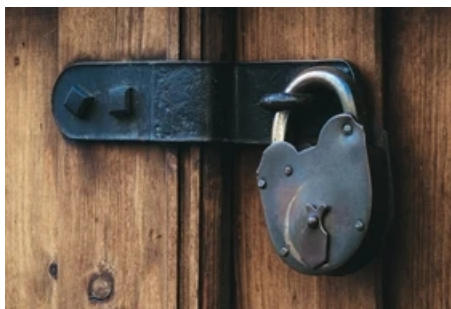
Comme nous le verrons en classe, il est assez simple d'imaginer, voir de mettre en place, des plans mettant à profit les vulnérabilités énoncées au point précédent :

- Il est inutile de mettre un verrou digital sur une vieille porte dont le mécanisme permet l'ouverture avec un déchet en plastique (exemple en classe).
- Pour les locaux demandant un niveau de sécurité élevé (médicaments dangereux, matériel informatique critique, etc.) les portes modernes combinées à un lecteur de badge ne suffisent pas toujours à prévenir d'une attaque inventive sur le mécanisme d'ouverture / de fermeture de porte.
- Un badge d'accès ça s'oublie, ça se perd, ça se prête et surtout ça se copie. Même si le personnel est sensibilisé correctement à la gestion responsable de son badge d'accès, on n'est jamais à l'abri d'une erreur d'inattention ou d'une personne qui utilise l'ingénierie sociale pour arriver à ses fins.
- La police nous surveille, mais qui surveille la police ? Les armoires à médicament ont des serrures, les tiroirs à documents aussi, la boîte à clé pour les armoires et les tiroirs à une serrure, mais qui a la clé de la boîte ? Personne car il faut trop souvent accéder aux clés alors le personnel laisse constamment la boîte ouverte en se disant « c'est pas grave il y a déjà la porte avec le badge d'accès ».

2.4. Démonstration de contre-mesures

Pour les quatre points ci-dessus, voici des contre-mesures simples et efficaces :

- Privilégier un verrou mécanique ou digital qui viennent en surcouche de sécurité du mécanisme initial pour les vieilles portes. On préfère ça :



Que ça :



- Protéger les bas et haut de porte pour éviter l'insertion de dispositifs permettant d'ouvrir la poignée de l'intérieur.
- Sensibiliser tout nouveau personnel aux badges d'accès et faire une formation de rappel annuel. Augmenter le niveau de sécurité des badges.
- Idéalement, utiliser des verrous électroniques, sinon, forcer le personnel qui doit détenir ces clés à les porter avec leur badge par exemple.

2.5. Limite de ces contre-mesures

Quelqu'un de déterminer à rentrer dans un espace donné finira toujours par y rentrer. La preuve en est qu'il existe encore de nos jours des cambriolages réussis de chambre fortes bancaires, de musées ou de bijouteries *high-end*. Un cadenas sur une porte ça se disque, une porte bien verrouillée, protégée en haut et en bas, ça s'enfonce au bélier. Un membre du personnel bien formé ouvrira une porte sous la menace ou sous l'effet d'une manipulation ingénieuse.

Il n'y a donc pas de système de sécurité physique parfait mais on peut pousser ceux-ci au maximum de leurs capacités pour un budget donné. Ce dernier doit être proportionnel aux risques encourus en cas de brèche. Mais, dans le cas des exemples donné ci-dessus, l'investissement à faire pour sécuriser est relativement minime comparé à la sécurité que ça apporte. Disons qu'un bas et un haut de porte en métal dur soient vissés depuis l'intérieur de la salle des médicaments. Le coût de l'opération s'élèverait à une cinquantaine d'euros et 15-20 minutes de travail, or le coût de la perte d'une armoire remplie de médicaments serait bien plus grand.

2.6. Approche pour l'institutionnalisation

Ici on vient répondre à certains éléments des sous-catégories Protect (CyFun 2025) pour acteurs « Important » suivantes :

- *PR.AA-06 : Physical access to assets is managed, monitored, and enforced commensurate with risk*
- *PR.AT-01 : Personnel are provided with awareness and training so that they possess the knowledge and skills to perform general tasks with cybersecurity risks in mind*

2.6.1. Politique

Notre nouvelles politique sera :

Toute porte donnant accès à des données ou matériel sensibles au sein de PVDV doit être sécurisée au maximum. Ce qui implique qu'il ne doit pas être possible de l'ouvrir avec des moyens rudimentaires. Les portes pour locaux sensibles doivent être rénovée et contenir un lecteur de badge hautement sécurisé. Les portes de locaux moins sensibles mais nécessitant néanmoins un verrou doivent garantir un niveau de sécurité minimum. Les clés et badges qui ouvrent ces portes doivent être gérés avec la plus grande prudence et responsabilité par leurs détenteurs. Le personnel sera formé et sensibiliser aux risques impliqués par les mauvaises pratiques en terme de cybersécurité physique et matérielle.

2.6.2. Procédures

Les procédures pour ces contre-mesures découlent des démonstrations : sensibiliser régulièrement le personnel, appliquer des mesures de sécurité physique adéquates et vérifier leurs bon fonctionnement au quotidien, document toute irrégularité, etc.

3. Étape 2 : une attaque de type « *BadUSB* »

3.1. Contexte et contre-mesures

Un autre hôpital témoigne avoir été victime d'un sabotage dont ils ont pu tracer l'origine, un périphérique HID (*Human Interface Device*) de type clavier, considéré comme légitime, aurait exécuté une série de commandes menant à la disruption des connexions aux serveurs distant du groupe Shirocks (contenant les données de rendez-vous). Pour pallier ce problème, l'équipe d'ingénieurs en cybersécurité du groupe Shirocks a pour mission de mettre en place des outils permettant de **filtrer les périphériques HID non-autorisés et d'être alerté si des tentatives avaient lieu**.

3.2. Configuration

Sachant qu'il faut attaquer **physiquement** un ordinateur, j'ai choisi d'utiliser un Raspberry Pi 4 avec Raspberry Pi OS (version *Desktop*) installé dessus. Ce faisant, j'aurai **l'équivalent d'un vrai ordinateur** (du type bureautique) sur lequel je peux installer un outil qui permettra d'**empêcher l'attaque via HID** (*USBGuard*) ainsi qu'un outil qui permettra de **détecter, alerter et journaliser les tentatives d'attaques** (Wazuh-Agent). Ce dernier envoie ses logs à un serveur distant (type SIEM) dans lequel tourne un *stack* Wazuh conteneurisé grâce à Docker Compose. Enfin, cet agent Wazuh aura la capacité de **déclencher un script si une alerte à lieu** (par exemple pour bloquer un périphérique USB non-autorisé qui aurait été oublié dans les règles de USBGuard).

USBGuard est un logiciel qui interagit directement avec le sous système USB du noyau Linux. Il permet d'appliquer une **politique de refus par défaut**, à la manière des *access-lists* d'un pare-feu. Lorsqu'un périphérique est inséré, le noyau le reconnaît mais ne charge pas les pilotes et ne permet aucune interaction (*Input/Output*) tant que le démon USBGuard n'a pas autorisé le périphérique **basé sur un ensemble de règles complexes** (incluant le *vendor ID*, le *product ID*, les hachages de descripteurs, les ports spécifiques, etc.).

3.2.1. Résumé des étapes de configuration

À des fins de lisibilité, je mentionnerai ici de manière synthétique les grandes étapes par lesquelles je suis passé pour arriver à mes fins. Les détails de configurations se trouvent ici ([USBGuard](#)) et ici ([Wazuh](#)).

- **Installation de USBGuard** sur la machine à protéger (Raspberry Pi 4 – Ordinateur fictif)
- **Configuration de plusieurs ACL USBGuard** afin de montrer trois états dans la démo : sans protection, avec protection et quand on contourne la protection.
- **Installation du SIEM Wazuh** sur ma machine personnelle (serveur SIEM administrateur fictif) via Docker Compose.
- **Installation du Wazuh Agent en *bare-metal*** sur la machine à protéger.
- **Création d'un parser personnalisé en python pour l'envoi des logs USBGuard** au SIEM Wazuh. En effet, les logs produit par USBGuard ne sont pas nativement pris en charge par Wazuh.
- **Configuration du SIEM Wazuh et du Wazuh Agent** afin de mettre en place la fonctionnalité Wazuh « *active response* ». Celle-ci permettra d'exécuter, si une règle spécifique est déclenchée, un script sur la machine où se trouve le Wazuh Agent. Dans mon cas, je choisis une mesure extrême qui éteint complètement et directement la machine si USBGuard témoigne dans ses logs avoir bloqué un périphérique USB.

3.2.2. Installation de USBGuard

ATTENTION ! Dans mon cas USBGuard a créé, dès l'installation, une liste de règles basée sur les périphériques USB déjà connectés. Si votre souris ou votre clavier est débranché au moment de l'installation, vous vous retrouverez bloqué hors de votre machine. Dans le cas d'un ordinateur portable, le clavier et le *touchpad* sont souvent reliés aux bus internes PS/2, I2C ou SPI. USBGuard n'aura donc aucun effet sur eux. Pour vérifier quels périphériques sont considérés comme USB avant d'installer USBGuard, exécutez la commande « **lsusb** ».

```
sudo apt-get update
```

```
sudo apt-get install usbguard
```

Avant d'utiliser USBGuard, on peut analyser au niveau du sous-système USB du noyau Linux les interactions qui seront interceptées par USBGuard grâce à :

```
udevadm monitor --kernel --subsystem-match=usb
```

```
doc-pavi-jilan@NotVerySecure-PC:~$ udevadm monitor --kernel --subsystem-match=usb
monitor will print the received events for:
KERNEL - the kernel uevent

KERNEL[3291.830362] add      /devices/platform/scb/fd500000.pcie/pci0000:00/0000:00:00.0/0000:01:00.0/usb1/1-1/1-1.4 (usb)
KERNEL[3291.830575] bind     /devices/platform/scb/fd500000.pcie/pci0000:00/0000:00:00.0/0000:01:00.0/usb1/1-1/1-1.4 (usb)
KERNEL[3291.840264] add      /devices/platform/scb/fd500000.pcie/pci0000:00/0000:00:00.0/0000:01:00.0/usb1/1-1/1-1.4/1-1.4:1.0 (usb)
KERNEL[3291.981125] bind     /devices/platform/scb/fd500000.pcie/pci0000:00/0000:00:00.0/0000:01:00.0/usb1/1-1/1-1.4/1-1.4:1.0 (usb)
KERNEL[3297.238060] unbind   /devices/platform/scb/fd500000.pcie/pci0000:00/0000:00:00.0/0000:01:00.0/usb1/1-1/1-1.4/1-1.4:1.0 (usb)
KERNEL[3297.239613] remove   /devices/platform/scb/fd500000.pcie/pci0000:00/0000:00:00.0/0000:01:00.0/usb1/1-1/1-1.4/1-1.4:1.0 (usb)
KERNEL[3297.253780] unbind   /devices/platform/scb/fd500000.pcie/pci0000:00/0000:00:00.0/0000:01:00.0/usb1/1-1/1-1.4 (usb)
KERNEL[3297.253961] remove   /devices/platform/scb/fd500000.pcie/pci0000:00/0000:00:00.0/0000:01:00.0/usb1/1-1/1-1.4 (usb)
```

On voit bien sur la machine du Docteur Pavi Jilan (le Raspberry Pi) que lorsque je branche et débranche le clavier, un évènement apparaît instantanément (dès que le signal électrique est reçu). Ce sont ces évènements qui seront interceptés par USBGuard afin qu'il applique ses règles dessus.

Voici les options proposées par USBGuard :

```
doc-pavi-jilan@NotVerySecure-PC:~$ usbguard --help
Usage: usbguard [OPTIONS] <command> [COMMAND OPTIONS] ...

Options:
  -h, --help            display this help message and exit
  -V, --version          display version number and exit

Commands:
  get-parameter <name>      Get the value of a runtime parameter.
  set-parameter <name> <value> Set the value of a runtime parameter.
  list-devices              List all USB devices recognized by the USBGuard daemon.
  allow-device <id|rule|p-rule> Authorize a device to interact with the system.
  block-device <id|rule|p-rule> Deauthorize a device.
  reject-device <id|rule|p-rule> Deauthorize and remove a device from the system.

  list-rules                List the rule set (policy) used by the USBGuard daemon.
  append-rule <rule>        Append a rule to the rule set.
  remove-rule <id>          Remove a rule from the rule set.

  generate-policy           Generate a rule set (policy) based on the connected USB devices.
  watch                    Watch for IPC interface events and print them to stdout.
  read-descriptor           Read a USB descriptor from a file and print it in human-readable form.

  add-user <name>           Add USBGuard IPC user/group (requires root privileges)
  remove-user <name>        Remove USBGuard IPC user/group (requires root privileges)
```

La bonne pratique pour générer une politique USBGuard (liste de règles) est de la générer dans un fichier délocalisé du dossier de configuration USBGuard :

```
usbguard generate-policy > rules.conf
```

Ensuite, il faut lire cette liste de règles afin de s'assurer qu'elle fait bien ce qu'on attend d'elle.

Pour finalement l'appliquer dans le dossier de configuration dédié :

```
sudo install -m 0600 -o root -g root rules.conf /etc/usbguard/rules.conf
```

Et redémarrer le démon pour qu'il prenne les modifications en compte :

```
sudo systemctl restart usbguard
```

On exécute l'opération sur l'ordinateur de Docteur Jilan, avec uniquement sa souris et son clavier connectés via USB :

```
doc-pavi-jilan@NotVerySecure-PC:~$ usbguard generate-policy > rules.conf
doc-pavi-jilan@NotVerySecure-PC:~$ nano rules.conf
doc-pavi-jilan@NotVerySecure-PC:~$ sudo install -m 0600 -o root -g root rules.conf /etc/usbguard/rules.conf
doc-pavi-jilan@NotVerySecure-PC:~$ sudo systemctl restart usbguard
```

Élément scénaristique : Il s'avère que le technicien qui met en place USBGuard sur les ordinateurs de la clinique a remarqué que certains claviers HP, portants exactement le même numéro de modèle, ont dû changer de microcontrôleur ou *firmware*, ce qui change le *hash* et rend compliqué le déploiement automatisé car ces claviers « divergents » se retrouvent bloqués. Il décide alors de rajouter une règle (encadrée en rouge ci-dessous) plus généraliste dans le fichier de règles. Elle autorise tous les claviers de ce modèle précis de la marque HP (grâce à l'ID, encadré en vert), peu importe le *hash* :

```
allow id 1d6b:0002 serial "0000:01:00.0" name "xHCI Host Controller" hash "WLLK8WnDcoZsmC13ldzWRmahoyPD7Y+TLn1uJ4TB2GU=" parent-hash "MvRVALwPgWjIPJ4ZsIUCR65B"
allow id 1d6b:0003 serial "0000:01:00.0" name "xHCI Host Controller" hash "CqYue16iqZ5kcyvggFg0VGT5tAS5PTaAH6XZ1EApa1w=" parent-hash "MvRVALwPgWjIPJ4ZsIUCR65B"
allow id 2109:3431 serial "" name "USB2.0 Hub" hash "/yzg/BjU0j0CTc2DYFSrSSr2JGwmQuhhAhpqTsXZA4=" parent-hash "WLLK8WnDcoZsmC13ldzWRmahoyPD7Y+TLn1uJ4TB2GU="
allow id 045e:00f0 serial "" name "Microsoft \xc2\xae Laser Mouse 6000" hash "8DBsCYtOxfMTSns7v7gR9q73j1g7yLM653vHsw62h+E=" parent-hash "/yzg/BjU0j0CTc2DYFSrSSr2JGwmQuhhAhpqTsXZA4="
allow id 03f0:0324 serial "" name "HP Basic USB Keyboard" hash "0ewRYEAPDUXKIJ3zGmKl7vLPl7recfxRV074w3qhRpw=" parent-hash "/yzg/BjU0j0CTc2DYFSrSSr2JGwmQuhhAhpqTsXZA4="
```

Les trois premières règles font référence aux *hubs* et contrôleurs USB qui sont aussi branché sur le bus USB. Si ils sont bloqués dans les règles, n'importe quel périphérique même autorisé ne pourra pas interagir avec la machine.

3.2.3. Installation de Wazuh

Afin que l'ordinateur du Docteur Jilan soit surveillé par le SIEM Wazuh de Shirocks, on installe sur son ordinateur un Wazuh Agent. C'est un HIDS (*Host Intrusion Detection System*), il sera chargé de surveiller tout le système de l'ordinateur afin de détecter d'éventuelles anomalies. Tous les événements qu'il génère sont envoyés à un *stack* conteneurisé Wazuh local à PVDV, puis renvoyé au serveur Wazuh du groupe Shirocks. Ce *stack* Docker est constitué du Wazuh Manager (reçoit les logs, déclenche les alertes et déclenche l'*active-response* si configuré), qui parse et stocke tout ces logs dans la base de données Wazuh Indexer, servant elle-même de source pour l'affichage de ces données dans le Wazuh Dashboard. Pour faire tourner le *stack* Wazuh Docker (en *single node*), il suffit de suivre les instructions de leur [documentation en ligne](#). Et pour installer le Wazuh Agent sur le Raspberry Pi, on suit d'autres instructions de leur [documentation en ligne](#). Ensuite, on modifie légèrement `/single-node/config/wazuh_cluster/wazuh_manager.conf` afin d'y ajouter :

```
<!-- Commande pour lancer le script d'extinction du pc de doc-pav-jilan -->
<command>
  <name>win-shutdown</name>
  <executable>force-shutdown.sh</executable>
  <timeout_allowed>no</timeout_allowed>
</command>
<!-- Active response si un blocage USB est detecte sur doc-pavi-jilan -->
<active-response>
  <command>win-shutdown</command>
  <location>local</location>
  <rules_id>100201</rules_id>
</active-response>
```

On crée la règle **100201** dans l'onglet Rules en modifiant le fichier **local_rules.xml** du Wazuh Manager :



Et du côté Wazuh Agent, on crée le script :

```
sudo nano /var/ossec/active-response/bin/force-shutdown.sh
```

Qui contient tout simplement :

```
#!/bin/bash
```

```
# Script de réponse active : extinction immédiate
```

```
# Commande d'extinction (nécessite sudo sans mot de passe)
```

```
sudo /usr/sbin/shutdown -h now
```

On lui donne les bons droits d'accès et permissions :

```
sudo chmod 750 /var/ossec/active-response/bin/force-shutdown.sh
```

```
sudo chown root:wazuh /var/ossec/active-response/bin/force-shutdown.sh
```

Et on édite les droits sudo pour donner à l'utilisateur wazuh (qui exécutera le script) le droit du privilège sudo **uniquement pour la commande de *shutdown*** :

```
sudo visudo
```

Et on ajoute la ligne :

```
wazuh ALL=(ALL) NOPASSWD: /usr/sbin/shutdown
```

Reste plus qu'à créer un parseur pour les logs de USBGuard, car ils ne sont pas nativement pris en charge par Wazuh. Pour ce faire, n'ayant pas les compétences en codage, j'ai utilisé l'IA Gemini pour générer un script python qui récupère les logs et les parse correctement en JSON. Ensuite j'en ai fait un service systemd pour la persistance.

Voilà ! Tout est bien configuré. Il faut bien redémarrer proprement les conteneurs Docker et le service systemd wazuh-agent pour que tout fonctionne.

3.3. Démonstration d'une menace potentielle

Plusieurs types de menaces existent en ce qui concerne les ports physique d'une machine. Que ce soit branché dans le port lui même ou sur les lignes internes (à la machine) et externes qui y mènent, l'utilisation de certains outils matériels permettent la surveillance, la récupération, la suppression ou l'injection de données. Le [site web](#) de Hak5 en est la preuve. Il n'existe d'ailleurs pas que des outils USB. Chez Hak5 on retrouve des intercepteurs Ethernet, HDMI voir même Thunderbolt. La plupart de ces outils sont plutôt coûteux, et demande un tantinet de compréhension technique (pour des attaques sophistiquées en tous cas).

Pour l'attaque dont je vais faire la démonstration il existe une alternative peu chère. Certes, elle est moins *user friendly* et un peu moins fonctionnelle dans son état initial (nécessite des connaissances poussées en électronique et MicroPython pour des fonctionnalités avancées) mais entièrement configurable car *open-source*, il s'agit du PicoUSB. Malheureusement, pour une partie justement plus poussée de l'attaque, je nécessite un élément en plus : **pouvoir falsifier mon descripteur USB**.

Le descripteur USB c'est un peu comme une carte d'identité que tout appareil USB fourni à une machine lors de sa connexion au port. On y retrouve diverses informations :

- L'identifiant, soit Vendor ID : Product ID (VID:PID – abcd:1234) ;
- Le nom en toute lettre du fabricant et du modèle ;
- Le firmware ;
- Les drivers acceptés ;
- etc.

J'utiliserai donc un Flipper Zero (couteau-suisse du *hacker*) qui intègre la fonctionnalité BadUSB avec comme langage d'écriture pour les scripts une version améliorée du Rubber Ducky Script (de Hak5). C'est grâce à cette amélioration que je pourrais contourner une règle (mal configurée) de USBGuard.

Afin de faire le lien avec la troisième étape et ses contre-mesures, j'ai décidé de créer une simulation (assez réaliste) de rançongiciel. Le Flipper Zero en mode BadUSB va ouvrir un terminal afin d'y écrire et exécuter un script qui chiffrera un dossier de démo (qui représente potentiellement toute la machine), et ce sans avoir besoin de privilèges ni d'installer des outils sur la machine. Vous trouverez le script en version « Rubber Ducky Script » et en version « Bash » en annexe de ce rapport.

3.4. Démonstration du fonctionnement de la contre-mesure

Je commence par lister les périphériques reconnus par USBGuard alors qu'il n'y a que la souris Microsoft et le clavier HP qui sont branchés. On voit bien que USBGuard les reconnaît (souligné en vert) et leur a appliqué les règles créées plus tôt, ils sont ainsi autorisés à communiquer sur le bus USB.

```
doc-pavi-jilan@NotVerySecure-PC:~$ usbguard list-devices
7: allow id 1d6b:0002 serial "0000:01:00.0" name "xHCI Host Controller" hash "WLLK8WnDc0ZsmC13ldzWRmahoyPD7Y+TLn1uJ4TB2GU=" parent-hash "MvRvALwPgWJIPJ4ZsIUCR
65FUMV4N8+5Xl/zyLmE9z4=" via-port "usb1" with-interface 09:00:00 with-connect-type ""
8: allow id 1d6b:0003 serial "0000:01:00.0" name "xHCI Host Controller" hash "CqYue16iqZ5kcyvqgFg0VGT5tAS5PTaAH6XZ1EAp1w=" parent-hash "MvRvALwPgWJIPJ4ZsIUCR
65FUMV4N8+5Xl/zyLmE9z4=" via-port "usb2" with-interface 09:00:00 with-connect-type ""
9: allow id 2199:3431 serial "" name "USB2.0 Hub" hash "/yzg/BjU0j0CTc2DYFSrSsr2JGwmQuhhAhpqTsXZA4=" parent-hash "WLLK8WnDc0ZsmC13ldzWRmahoyPD7Y+TLn1uJ4TB2GU
=" via-port "1-1" with-interface 09:00:00 with-connect-type "unknown"
17: allow id 045e:00f0 serial "" name "Microsoft \xc2\xae Laser Mouse 6000" hash "8DBsCYtoxfMTSns7v7gR9q73j1g7yLM653vHsw62h+E=" parent-hash "/yzg/BjU0j0CTc2DY
FSrSsr2JGwmQuhhAhpqTsXZA4=" via-port "1-1.2" with-interface 03:01:02 with-connect-type "unknown"
18: allow id 03f0:0324 serial "" name "HP Basic USB Keyboard" hash "0ewRYEAPDUXKI3ZGmKl7vLPl7reCfxRV074w3qhrpw=" parent-hash "/yzg/BjU0j0CTc2DYFSrSsr2JGwmQuh
hAhpqTsXZA4=" via-port "1-1.4" with-interface 03:01:01 with-connect-type "unknown"
```

Ensuite, je branche sur mon ordinateur personnel un PicoUSB configuré pour ouvrir un terminal et écrire dedans et ça fonctionne :

```
# thruya @ pop-os in ~ [18:09:19]
$ THIS PC HAS BEEN HACKED --- THIS PC HAS BEEN HACKED ---
THIS PC HAS BEEN HACKED --- THIS PC HAS BEEN HACKED --- TH
IS PC HAS BEEN HACKED --- THIS PC HAS BEEN HACKED --- THIS
PC HAS BEEN HACKED ---THIS PC HAS BEEN HACKED --- THIS PC
HAS BEEN HACKED --- THIS PC HAS BEEN HACKED --- |
```

Je fais pareil sur la machine USBGuard mais rien ne se passe. J'essaye alors avec mon Flipper Zero (sans configuration spéciale) et rien non plus. C'est normal, ils sont correctement bloqués par USBGuard ! On peut le vérifier en listant à nouveau les périphériques. **On y voit bien le Flipper et le Pico correctement bloqués.**

```
doc-pavi-jilan@NotVerySecure-PC:~$ usbguard list-devices
7: allow id 1d6b:0002 serial "0000:01:00.0" name "xHCI Host Controller" hash "WLLK8WnDc0ZsmC13ldzWRmahoyPD7Y+TLn1uJ4TB2GU=" parent-hash "MvRvALwPgWJIPJ4ZsIUCR
65FUMV4N8+5Xl/zyLmE9z4=" via-port "usb1" with-interface 09:00:00 with-connect-type ""
8: allow id 1d6b:0003 serial "0000:01:00.0" name "xHCI Host Controller" hash "CqYue16iqZ5kcyvqgFg0VGT5tAS5PTaAH6XZ1EAp1w=" parent-hash "MvRvALwPgWJIPJ4ZsIUCR
65FUMV4N8+5Xl/zyLmE9z4=" via-port "usb2" with-interface 09:00:00 with-connect-type ""
9: allow id 2199:3431 serial "" name "USB2.0 Hub" hash "/yzg/BjU0j0CTc2DYFSrSsr2JGwmQuhhAhpqTsXZA4=" parent-hash "WLLK8WnDc0ZsmC13ldzWRmahoyPD7Y+TLn1uJ4TB2GU
=" via-port "1-1" with-interface 09:00:00 with-connect-type "unknown"
10: allow id 045e:00f0 serial "" name "Microsoft \xc2\xae Laser Mouse 6000" hash "8DBsCYtoxfMTSns7v7gR9q73j1g7yLM653vHsw62h+E=" parent-hash "/yzg/BjU0j0CTc2DY
FSrSsr2JGwmQuhhAhpqTsXZA4=" via-port "1-1.2" with-interface 03:01:02 with-connect-type "unknown"
11: allow id 03f0:0324 serial "" name "HP Basic USB Keyboard" hash "0ewRYEAPDUXKI3ZGmKl7vLPl7reCfxRV074w3qhrpw=" parent-hash "/yzg/BjU0j0CTc2DYFSrSsr2JGwmQuh
hAhpqTsXZA4=" via-port "1-1.4" with-interface 03:01:01 with-connect-type "unknown"
12: block id 0483:5740 serial "flip oldoond" name "Flipper oldoond" hash "4I0LJ6SJuhYGMmfZvfg/+J96ZEA/ESNOTLVVAY6Mc2o=" parent-hash "/yzg/BjU0j0CTc2DYFSrSsr
2JGwmQuhhAhpqTsXZA4=" via-port "1-1.1" with-interface { 02:02:00 0a:00:00 } with-connect-type "unknown"
13: block id 239a:80f4 serial "E66368254F37A427" name "Pico" hash "PHW1aHifnEvPkJ2tiL6Wvz0ud5nB332TVb4AWEZPBFY=" parent-hash "/yzg/BjU0j0CTc2DYFSrSsr2JGwmQuhh
AhpqTsXZA4=" via-port "1-1.3" with-interface { 02:02:00 0a:00:00 03:00:00 01:01:00 01:03:00 } with-connect-type "unknown"
```

Maintenant, grâce à Wazuh, il nous est possible de détecter qu'un périphérique a été bloqué par USBGuard. Grâce à l'*active response* j'ai mis en place une mesure extrême (mais efficace pour la démo) : le *shutdown* instantané dès qu'un blocage est détecté. **Je ne peux pas faire de capture d'écran très parlante pour ce rapport mais vous verrez le script en action lors de la démonstration en direct.**

3.5. Limites de la contre-mesure

Si les règles de la liste d'accès sont trop laxes (par exemple, tous les claviers de la marque HP sont autorisés), il est facile pour un attaquant de se faire passer pour un appareil spécifique. Le technicien qui a inscrit la règle généraliste autorisant tous les claviers HP d'un même modèle a fait une grosse erreur, un clavier malveillant s'est connecté avec succès.

```
doc-pavi-jilan@NotVerySecure-PC:~$ usbguard list-devices
7: allow id 1d6b:0002 serial "0000:01:00:0" name "xHCI Host Controller" hash "WLLK8WnDc0ZsmC13ldzWRmahoyPD7Y+TLn1uJ4TB2GU=" parent-hash "MvRvALwPgWJIPJ4ZsIUcR
65FuMV4N8+5X1/ZylmE9z4=" via-port "usb1" with-interface 09:00:00 with-connect-type ""
8: allow id 1d6b:0003 serial "0000:01:00:0" name "xHCI Host Controller" hash "CqYue16iqZ5kcyvqgFg0V6T5tAS5PTaAH6XZ1EApaiw=" parent-hash "MvRvALwPgWJIPJ4ZsIUcR
65FuMV4N8+5X1/ZylmE9z4=" via-port "usb2" with-interface 09:00:00 with-connect-type ""
9: allow id 2109:3431 serial "" name "USB2.0 Hub" hash "/yzg/Bju0j0CtC2DYFSrSr2JGwmQuhhAhpqcTsXZA4=" parent-hash "WLLK8WnDc0ZsmC13ldzWRmahoyPD7Y+TLn1uJ4TB2GU
=" via-port "1-1" with-interface 09:00:00 with-connect-type "unknown"
10: allow id 045e:00f0 serial "" name "Microsoft \xc2\xae Laser Mouse 6000" hash "8DBsCYt0xfMTSNs7v7gR9q73j1g7ylM653vHsw62h+E=" parent-hash "/yzg/Bju0j0CtC2DY
FSrSr2JGwmQuhhAhpqcTsXZA4=" via-port "1-1.2" with-interface 03:01:01 with-connect-type "unknown"
11: allow id 03f0:0324 serial "" name "HP Basic USB Keyboard" hash "0ewRYEAPDUXKIJ3zGmKl7vLpL7reCfXRv074w3qhRpw=" parent-hash "/yzg/Bju0j0CtC2DYFSrSr2JGwmQuh
hAhpqcTsXZA4=" via-port "1-1.4" with-interface 03:01:01 with-connect-type "unknown"
16: allow id 03f0:0324 serial "" name "FakeHPKeyboardWithBadIntentions" hash "qKbPVnAbFEt0bzMocxeMk/LgsYTqCCa65QNuN8ABgSc=" parent-hash "/yzg/Bju0j0CtC2DYFSrS
r2JGwmQuhhAhpqcTsXZA4=" via-port "1-1.1" with-interface 03:01:01 with-connect-type "unknown"
```

Pourquoi ? Comment ? Car la notion de **hash** est très importante dans USBGuard. On voit bien ci-dessus que le *hash* n'est pas le même pour les deux claviers. Si une règle USBGuard ne mentionne pas le *hash*, elle laisse passer tout périphérique qui déclare avoir le VID:PID requis. Mais le problème c'est que même avec le *hash* bien configuré dans les règles il est encore possible de se faire passer pour un autre. En effet, les revendeurs de matériel « bas de gamme » (clavier, souris, etc.) n'utilise que très rarement, dans le descripteur USB, le champ **iSerial** (pour des raisons pécuniaires bien sûr). Ce champ permet de donner un numéro de série à chaque périphérique, rendant ainsi le *hash* différent pour chacun. Donc, si un attaquant achète le même modèle de clavier que vous et qu'il arrive à copier toutes les informations du descripteur USB, il peut se faire passer pour votre clavier avec **exactement le même hash**. Il y a deux solutions pour pallier ce problème. La première, dans les règles USBGuard, empêcher l'utilisation d'un deuxième clavier tant qu'un premier est branché (c'est possible apparemment), et s'assurer de brancher le seul clavier dans un port difficilement accessible physiquement pour l'attaquant. La deuxième, si l'attaquant arrive à débrancher le premier clavier, serait de mettre en place une/des règle(s) Wazuh qui comprendrait que le clavier à été débranché puis rebranché afin de bloquer celui-ci. Idéalement, il faudrait combiner les deux pour le plus haut niveau de sécurité.

Le problème c'est que si ces règles sont trop restrictives, il y a des chances que les utilisateur·rice·s des ordinateurs où USBGuard est actif finissent par chercher des moyens contournés d'avoir ce qu'ils·elles veulent. Par exemple : si un clavier mal configuré dans USBGuard ne fonctionne plus du jour au lendemain, un·e docteur pourrait chercher à accéder à sa plateforme Shirocks sur un appareil personnel/privé, rendant ainsi la détection d'éventuelles attaques impossible (*Shadow IT*).

Il existe d'autres limitations qui sont principalement liées au périmètre que couvre USBGuard :

- Pour Wazuh, pas de détection ni d'*active-response* sur la machine où tourne le Wazuh Agent si le Wazuh Manager est inactif (pour mise à jour ou panne de courant par exemple).

- Ordinateur portable : clavier et touchpad rarement reconnus car passent par des bus internes (PS/2, SPI ou I2C).
- Dans le cas des ports USB-C, si le port est compatible Thunderbolt (USB4) il peut parler directement au bus PCIe et contourner ainsi la protection USBGuard. Cela pourrait permettre une attaque de type DMA (*Direct Memory Access*) où l'attaquant a accès directement à la mémoire RAM sans demander la permission au CPU. Cela étant dit, il existe un mécanisme Linux (IOMMU) et un outil appelé Bolt qui permettent de sécuriser cette vulnérabilité.
- Si un attaquant a accès au composant interne d'une machine, il peut y installer un appareil qui agira comme un MitM (*Man in the Middle*), par exemple sur les bus USB, I2C et PS/2 voir sur un port M.2 laissé libre.
- Si le bluetooth est mal configuré : le Flipper Zero permet aussi de lancer son script en mode « clavier bluetooth ».
- Enfin, dans la même logique, si un ordinateur dispose d'un logiciel de pilotage à distance, il ne sera pas protégé par USBGuard non plus.

3.6. Approche pour l'institutionnalisation de la contre-mesure

Je m'installe maintenant dans la peau de l'ingénieur qui a été chargé de mettre en place la contre-mesure :

Comme nous avons pu le constater précédemment, il existe un réel risque à laisser ses ports USB ouvert à tous et sans surveillance par un outil comme Wazuh. Cependant, il s'agit ici de rester réaliste par rapport aux probabilités de certains types d'attaque et les impacts réels engendrés. J'entends par là qu'il est nécessaire, comme toujours en cybersécurité, de réfléchir au coût en temps et en argent de la mise en place d'une telle contre-mesure, ainsi que son intégration dans les logiques métiers. Dans le cadre du secteur médical, il est d'autant plus important de réfléchir à ces éléments car il ne faut pas ralentir ou empêcher les activités du corps soignant. Ainsi, en coordination avec les corps métiers et le conseil administratif, nous décidons de mettre en place une politique interne impliquant la création de procédures qui permettront la mise en place concrète des moyens de protections adéquats et leur maintenance.

Les sous-catégories Protect (CyFun 2025) pour acteur « Important » auxquelles nous répondront ici sont les suivantes :

- *PR.AA-06 : Physical access to assets is managed, monitored, and enforced commensurate with risk*
- *PR.PS-04 : Log records are generated and made available for continuous monitoring*

Cela étant, les contre-mesures mises en place ici permettent aussi de répondre à des requis d'autres catégories CyFun comme Detect ou Identify (à travers Wazuh).

3.6.1. Politique interne

Toute machine (ordinateurs, serveurs, routeurs, etc.) présente sur le site de PVDV doit, tant que possible protéger, à l'aide de boîtiers sécurisés ou de bloqueur matériels, ses ports physiques (USB, Ethernet, etc.) d'accès non-autorisés. Pour les ports physiques ne pouvant être protégés de la sorte et ceux déjà protégés physiquement, il est obligatoire de mettre en place un/des outil(s) logiciels permettant de :

- Détecter et documenter les (dé)branchements sur lesdits ports ;
- Autoriser ou bloquer l'accès pour des périphériques spécifiques ;
- Activer une réponse de défense sur une machine compromise ;

3.6.2. Procédures (focus sur les port USB)

Pour garantir la pérennité de ladite politique :

- Jusqu'à nouvel ordre, le logiciel choisi pour la protection est USBGuard. L'installation d'USBGuard se fera par un technicien Shirocks qualifié sur toutes les machines de PVDV. Pour ce faire, il devra utiliser des outils d'automatisation tant que possible et portera une attention particulière à ne pas se verrouiller hors de la machine sur laquelle il travaille. Chaque machine se verra évaluer les périphériques USB essentiels dont elle a besoin pour fonctionner. Tout le reste devra être bloqué par défaut.
- Jusqu'à nouvel ordre, le logiciel choisi pour la détection et protection en temps réel est Wazuh. L'installation d'un Wazuh Agent se fera par un technicien Shirocks qualifié sur toutes les machines de PVDV. Ces agents enverront leurs logs au serveur SIEM Wazuh installé dans les serveurs locaux de PVDV. Ce dernier renverra ses logs au SIEM général de Shirocks.
- Dans le cadre d'un besoin spécifique concernant l'ajout, la suppression ou le changement d'un périphérique USB, le service IT de Shirocks devra systématiquement être contacté avant toute opération.
- Un petit stock de rechange de périphériques USB (clavier, souris, etc.) sera stocké dans un local sécurisé au cas où un remplacement doit avoir lieu. Ce stock devra être révisé annuellement.
- Shirocks doit conserver, dans un local sécurisé, au minimum un exemplaire de chaque périphérique USB de type HID (afin de reprendre le contrôle d'une machine dont le clavier ou la souris ne fonctionnerai plus). Un accès à distance bien configuré devra également être mis en place pour assister le personnel de la clinique en cas de panne.
- Les règles du SIEM concernant les périphériques USB doivent être mise à jour automatiquement. Elles doivent aussi être révisées manuellement chaque année.

4. Étape 3 : le rançongiciel

4.1. Contexte et contre-mesure

Un groupe criminel de *hackers*, les « *LockingLords* », poursuit actuellement sa campagne de rançongiciels visant à faire profit des infrastructures informatiques mal configurées des petites et moyennes cliniques médicales belges. Le groupe d'activistes anti-vaccins (mentionné [ici](#)) en profite pour faire croire au grand public que leur sabotage est en fait l'œuvre des *LockingLords*. Cette technique a été utilisée par (supposément) l'état Russe pour attaquer des cibles Ukrainiennes à des fins de sabotage. Le virus, maintenant bien connu, s'appelle « [NotPetya](#) ». Il se faisait passer pour un rançongiciel or son but ultime était tout simplement de détruire (supprimer) tout sur son passage, ce qu'on appelle un « *Wiper* ». Afin de protéger les bases de données locales des cliniques du groupe Shirocks, l'équipe d'ingénieurs en cybersécurité a été missionnée de réfléchir à l'implémentation de sauvegardes chiffrées et résilientes au rançongiciel « *LockedHealth 3.0* ».

4.2. Configuration

Pour rester dans un esprit de déploiement réel, j'utilise un **deuxième** Raspberry Pi 4 qui fera office de serveur de stockage des *backups* locaux. Ce serveur fait tourner un conteneur Docker du *framework* **MinIO**. Cet outil est une solution moderne et grandement utilisée pour des sauvegardes compatibles S3. Le modèle **S3 (Simple Storage Service)** est un standard de stockage orienté « objet », initialement popularisé par Amazon. Contrairement au stockage de fichiers classique, il organise les données en unités distinctes (objets) associées à des métadonnées riches. Cette architecture permet une gestion simplifiée via des requêtes HTTP et offre des fonctionnalités avancées de sécurité, comme le **verrouillage des données**, indispensables pour garantir l'intégrité face aux cybermenaces. MinIO supporte nativement le « *S3 Object Locking* » qui permet **l'immuabilité** des données sauvegardées. Les disques durs (une clé USB pour la démo), sont chiffrés avec LUKS afin de **garantir la protection des données au repos**.

J'adopte une architecture où les clients de sauvegarde utilisent un script type « cron » ou service systemd qui **chiffre et pousse les données via Restic** vers le serveur de sauvegarde MinIO. Ainsi, grâce à la fonction « *S3 Object Locking* » même si le client de sauvegarde est corrompu par un rançongiciel, il ne pourra que pousser de nouvelles sauvegardes ou lire les anciennes. Il sera **incapable** d'effectuer des modifications ou suppressions sur le serveur MinIO. Ce faisant, en cas de corruption totale du système informatique de PVDV, voir même de Shirocks, il existera toujours une version récente des données utiles prête à être utilisée. MinIO expose un port HTTP (RESTful) pour recevoir les données du client de sauvegarde, il n'y a donc aucune clé SSH à configurer, ce qui protège le système d'exploitation d'une tentative d'accès par SSH.

4.2.1. Résumé des étapes de configuration

À des fins de lisibilité, je mentionnerai ici de manière (relativement) synthétique les grandes étapes par lesquelles je suis passé pour arriver à mes fins. Les détails techniques utiles seront consultables lors de la démonstration en classe. Les scripts et fichiers utiles seront en annexe. Pour une vue plus « directe » (avec *screenshots*), rendez-vous dans la partie [« démonstration du fonctionnement cette contre-mesure »](#).

- **Installation de Ubuntu Server 25.10** sans interface graphique (*headless*) sur un deuxième Raspberry Pi 4.
- **Configuration des clés SSH** pour communiquer avec les différents comptes que je vais créer sur le serveur base de données et la machine cliente.
- **Installation de Docker** sur le serveur BDD pour y faire tourner MinIO.
- **Création et configuration du `docker-compose.yml`** pour MinIO.
- **Montage du dossier `/secure-backups`** de la clé USB (qui représente un disque dur dédié au serveur) sur `/mnt/minio-usb/secure-backups` et modification de `/etc/fstab` pour la **redondance du montage de partition au démarrage**.
- **Création d'un conteneur chiffré (.img) pour LUKS** : chiffrement, ouverture logique (mapping) et formatage interne du conteneur chiffré en « ext4 ».
- **Déchiffrement redondant du conteneur chiffré au démarrage de la machine** grâce à un fichier clé (*keyfile*) stocké sur la carte micro SD du Raspberry Pi MinIO.
- **Lancement du Docker Compose MinIO fonctionnel.**
- **Installation de l'outil en ligne de commande de MinIO : « mc »**
- **Configuration avec mc pour :**
 - **Créer un alias** au conteneur Docker MinIO (car mc peut gérer plusieurs instances MinIO à la fois).
 - **Créer un espace de stockage objet (*bucket*) sécurisé**, c'est à dire avec le verrouillage objet actif qui empêche la suppression.
 - **Instaurer la politique de rétention par défaut** pour tous les objets du *bucket* :
 - Le mode **COMPLIANCE** est choisi car il rend **impossible de réduire la durée de rétention ou de supprimer les fichiers**.
 - **La durée de rétention est de 30 jours** (1 jour pour le *bucket* de la démo).
- **Installation de Restic** sur la machine cliente et sur la machine MinIO.
- **Création d'un utilisateur « minio-pusher » sur la machine client.** Il dispose des moindres privilèges nécessaires à sa mission et utilise un mot de passe aléatoire à 30 caractères, stocké dans un gestionnaire de mot de passe du sysadmin.

- **Création d'un utilisateur « sysadmin » sur la machine client.** Il dispose de privilèges élevé car il doit pouvoir administrer la machine. Il utilise un mot de passe aléatoire à 30 caractères, stocké dans un gestionnaire de mot de passe du sysadmin.
- **Bridage des privilèges de l'utilisateur « doc-pavi-jilan »** au strict minimum nécessaire pour son usage de la machine (accès au bureau, à ses fichiers, etc.).
- **Configuration d'un réseau Tailscale** pour les intervenants au système de sauvegarde :
 - **Installation de Tailscale sur le serveur MinIO, la machine cliente et le serveur SIEM.** Ensuite, on intègre chacune de ces machines au réseau Tailscale grâce à `sudo tailscale up`.
 - **Configuration pare-feu du serveur MinIO et de la machine cliente** via un bash « script-firewall-final.sh ». **On bloque tout les ports pour le réseau LAN** et on autorise le trafic entrant et sortant de l'interface réseau Tailscale (tailscale0).
 - **Configuration des ACLs Tailscale** via des *tags* personnalisés :
 - On autorise l'admin (mon ordinateur personnel / le serveur SIEM) à accéder à tout les ports de toutes les machines.
 - On autorise les machines clientes de sauvegarde à accéder uniquement au port 9000 du serveur MinIO. Et on autorise le serveur MinIO et les machines clientes à avoir les ports 1514 et 1515 ouverts pour le Wazuh Agent.
- **Configuration de l'architecture de sauvegarde sécurisée :**
 - **Utilisation de Restic pour chiffrer et pousser les données de sauvegarde** depuis le compte **minio-pusher** vers le *bucket* dédié sur le serveur MinIO :
 - **Création d'un repo Restic** sur le *bucket*.
 - **Création d'un fichier caché de configuration d'identifiants** (`/home/minio-pusher/.restic-secrets`) dont minio-pusher est le propriétaire et le seul qui peut le lire.
 - Il utilise alors ces identifiants, qui lui sont propres sur le serveur MinIO.
 - **On applique une politique stricte à minio-pusher**, configurée depuis **mc** sur le serveur MinIO, afin qu'il ne soit **autorisé qu'à pousser ou lire des sauvegardes**.
 - Création d'un script de sauvegarde `/usr/local/bin/backup-pvdrv-secure.sh` et modification de **visudo** pour donner à **minio-pusher** des droits **sudo uniquement** pour ce script.
 - Création d'une tâche **cron** pour l'exécution de **backup-pvdrv-secure.sh** toute les heures entre 8h et 20h.

- **Utilisation de Restic pour avoir de la profondeur de *backup* grâce aux *snapshots* :**
 - De manière native, Restic compare la dernière sauvegarde (le dernier *snapshot*) avec celle qu'il s'apprête à faire. Il ne poussera dans le nouveau *snapshot* que ce qui a changé. Ainsi, **on peut choisir de restaurer les données dans un état bien précis.**
 - **On crée une politique de suppression des *snapshots* basée sur le temps** afin toujours avoir une sauvegarde « fraîche à disposition ». Le verrouillage objet natif S3 empêche toute suppression sur le *bucket*, il faut donc en prendre compte. Le double avantage c'est que ça évite tant les attaques de types *wiper* que les mauvaises manipulation d'un administrateur du serveur.
 - **Le script de nettoyage se trouve sur la machine serveur MinIO.** Il utilise les identifiants administrateurs du conteneur Docker MinIO et s'exécute régulièrement grâce à une tâche **cron**.
- **Utilisation de Restic pour restaurer une sauvegarde :**
 - L'utilisateur **minio-pusher** est restreint de lecture sur les données de sauvegarde. Il ne peut que pousser des données et lire les méta-données nécessaires au bon fonctionnement des *snapshots* Restic.
 - J'ai donc créé un compte utilisateur à **sysadmin** (via l'utilitaire **mc**) qui est spécifiquement conçu pour la restauration. Il a presque la même politique que **minio-pusher** avec l'accès au données de sauvegarde ajouté.
- **Configuration d'une surveillance en temps réel via Wazuh afin de vérifier l'intégrité des fichiers critiques** (sudoers, .restic-secrets, crontab, scripts de sauvegarde, etc.) :
 - Installation d'un Wazuh Agent sur la machine MinIO et configuration des *paths* à surveiller.
 - Configuration des chemins à surveiller par le Wazuh Agent sur la machine cliente.

4.3. Démonstration d'une menace potentielle

Comme démontré par les statistiques dans la partie du présent rapport sur le contexte d'entreprise, la menace numéro un pour le secteur de la santé est le rançongiciel. Ce type de menace prolifère particulièrement facilement dans le secteur de la santé dû au manque de sécurité des infrastructures informatiques. Un *ransomware* peut prendre plusieurs formes et, bien que son code soit parfois très complexe, l'exécution est souvent assez simple, ce qui le rend d'autant plus attrayant pour des groupes de *hackers* criminels dont le but est de rentabiliser chaque minute investie, à la manière d'une méga-corporation capitaliste. Le grand défi pour ces groupes est de réussir à atteindre le cœur du système informatique d'une organisation. Une fois cet objectif atteint, le rançongiciel fera assez de dégâts pour paralyser les activités de quasiment toute organisation, car la plupart repose sur un système d'information numérique. Cependant, dans leurs démarches avares, les groupes s'arrêtent parfois à la compromission d'un plus petit lot de données qui, bien qu'il ne paralyse pas l'organisation, permet de générer un petit pactole.

De fait, la démonstration que je vais faire est limitée, car je ne dispose pas encore de toutes les compétences me permettant de procéder à une escalade de privilège et/ou à un déplacement latéral, ni n'ai-je eu le temps de mettre en place un réseau réaliste de machine vérolées me le permettant.

Comme mentionné [plus haut dans ce rapport](#) la menace est, dans mon cas, un rançongiciel encodé et exécuté en directement dans le terminal, depuis un appareil BadUSB. Ce script, disponible en annexe, va chiffrer un répertoire spécifique. Ici, je lui dis de chiffrer `/home/doc-pavi-jilan/demo-minio`. Vous trouverez une courte vidéo qui démontre son fonctionnement en annexe de ce rapport : **ransomware-demo.mp4**

Voici les quatre lignes du script qui ont vraiment un effet, le reste, c'est du *show* :

```
TARGET_DIR="/home/doc-pavi-jilan/demo-minio"
```

```
ENC_FILE="$TARGET_DIR.locked.enc"
```

```
tar -czf - "$TARGET_DIR" 2>/dev/null | openssl enc -aes-256-cbc -salt -pbkdf2 -pass  
pass:hacker123 -out "$ENC_FILE"
```

```
rm -rf "$TARGET_DIR"
```

On mentionne manuellement le *path* de la ressource à chiffrer mais on pourrait imaginer un script de « découverte » qui testerai les permissions de l'environnement afin de chiffrer tout ce qui est à sa portée. On mentionne le *path* du dossier une fois chiffré. Ensuite la magie opère, on compresse `/demo-minio` en *gzip* vers la sortie standard, on passe cette sortie dans un pipe vers *openssl* qui va se charger du chiffrement du dossier compressé.

Le mot de passe de chiffrement est « hacker123 ». Idéalement, il faudrait l'encoder de manière interactive afin que le mot de passe ne soit pas récupérable avec une simple commande `ps aux`. De plus, il faudrait un autre script pour supprimer le script de rançongiciel qui contiendrait le mot de passe.

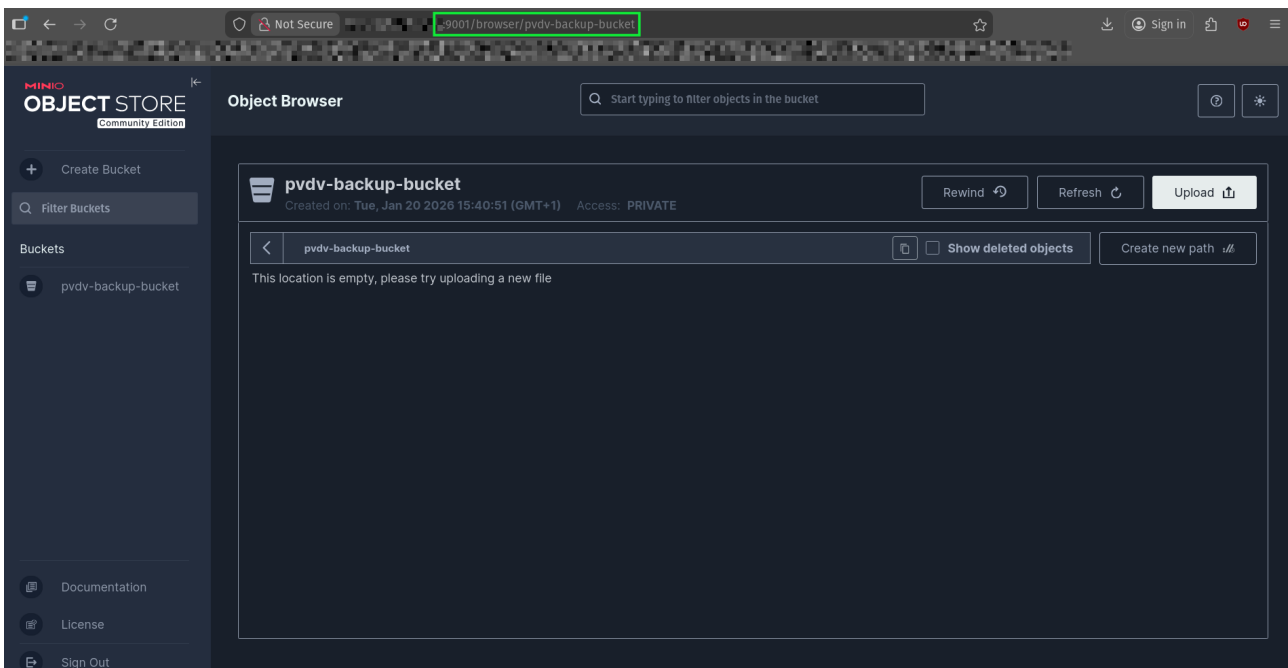
4.4. Démonstration du fonctionnement de la contre-mesure

Après avoir configuré le conteneur MinIO sur le Raspberry Pi, j'installe l'utilitaire en ligne de commande **mc** (*minio commandline*) pour créer mon premier objet de sauvegarde (*bucket*) en mode COMPLIANCE avec un délai de rétention de 30 jours. Je peux accéder au *bucket* via la GUI MinIO sur le port 9001 du serveur :

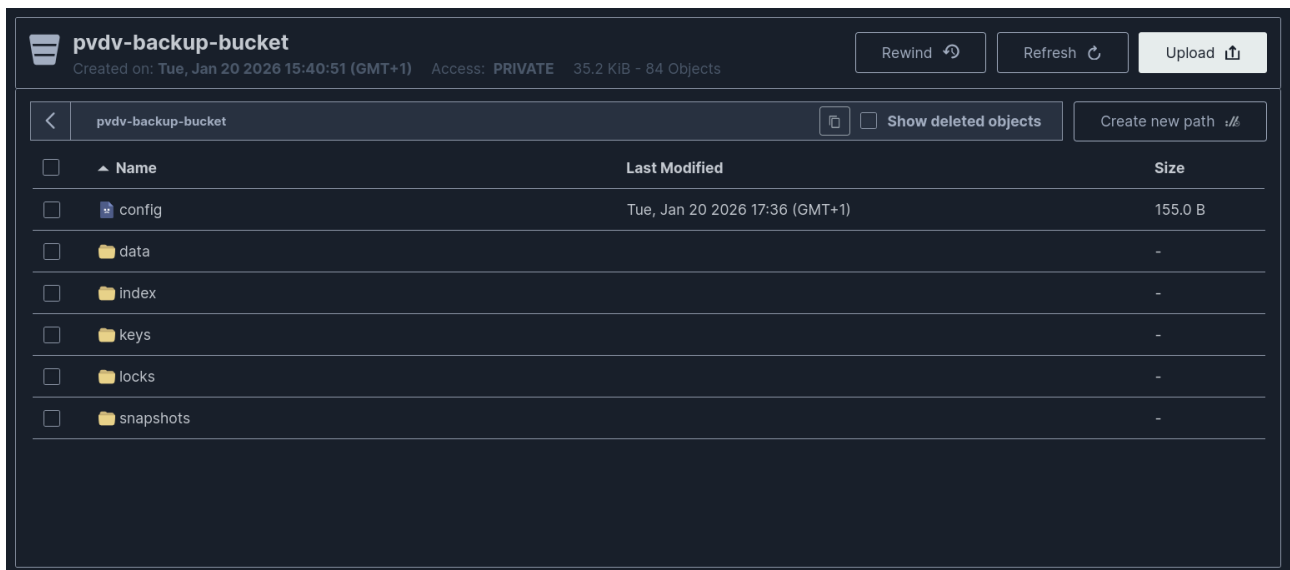
```
minio-admin@VerySecureBackup-DB:~/minio-docker$ mc --version
mc version RELEASE.2025-08-13T08-35-41Z (commit-id=7394ce0dd2a80935aded936b09fa12cbb3cb8096)
Runtime: go1.24.6 linux/arm64
Copyright (c) 2015-2025 MinIO, Inc.
License GNU AGPLv3 <https://www.gnu.org/licenses/agpl-3.0.html>
minio-admin@VerySecureBackup-DB:~/minio-docker$ mc alias set pvdv_minio http://localhost:9000 admin_super_secure complex
mc: Configuration written to `/home/minio-admin/.mc/config.json`. Please update your access credentials.
mc: Successfully created `/home/minio-admin/.mc/share`.
mc: Initialized share uploads `/home/minio-admin/.mc/share/uploads.json` file.
mc: Initialized share downloads `/home/minio-admin/.mc/share/downloads.json` file.
Added `pvdv_minio` successfully.
minio-admin@VerySecureBackup-DB:~/minio-docker$ mc mb pvdv_minio/pvdv-backup-bucket --with-lock
Bucket created successfully `pvdv_minio/pvdv-backup-bucket`.
minio-admin@VerySecureBackup-DB:~/minio-docker$ mc retention set --default compliance 30d pvdv_minio/pvdv-backup-bucket
Object locking 'COMPLIANCE' is configured for 30DAYS.
```

```
minio-admin@VerySecureBackup-DB:~/minio-docker$ mc stat pvdv_minio/pvdv-backup-bucket
Name       : pvdv-backup-bucket
Date       : 2026-01-20 15:47:48 CET
Size       : N/A
Type       : folder

Properties:
Versioning: Enabled
LockConfiguration:
  RetentionMode: COMPLIANCE
  Retention Until Date: 30DAYS
Location: us-east-1
Anonymous: Disabled
ILM: Disabled
```



Après avoir initialisé mon *repo* Restic au sein du *bucket* pvdv-backup-bucket, je peux y effectuer des *snapshots*. Vue du *repo* initié dans le *bucket* depuis la GUI :



Vue en CLI des snapshots effectués :

```
minio-pusher@NotVerySecure-PC:~ $ restic snapshots
repository 59c6f6d9 opened (version 2, compression level auto)
```

ID	Time	Host	Tags	Paths	Size
e1c903de	2026-01-20 19:08:23	NotVerySecure-PC	demo-pvdv-secure	/home/doc-pavi-jilan/demo-minio	341 B
d56ce54b	2026-01-23 14:17:03	NotVerySecure-PC	demo-pvdv-secure	/home/doc-pavi-jilan/demo-minio	0 B
e11fb3e2	2026-01-23 14:23:47	NotVerySecure-PC	demo-pvdv-secure	/home/doc-pavi-jilan/demo-minio	0 B
54644ba6	2026-01-23 14:32:38	NotVerySecure-PC	demo-pvdv-secure	/home/doc-pavi-jilan/demo-minio	651 B
e0c4a9a1	2026-01-23 14:52:54	NotVerySecure-PC	test-tag	/home/doc-pavi-jilan/demo-minio	651 B
5d54ff4f	2026-01-23 18:13:02	NotVerySecure-PC	auto-pvdv-secure	/home/doc-pavi-jilan/demo-minio	993 B
8a77f3aa	2026-01-23 18:14:01	NotVerySecure-PC	auto-pvdv-secure	/home/doc-pavi-jilan/demo-minio	993 B
6e18cba8	2026-01-23 18:15:01	NotVerySecure-PC	auto-pvdv-secure	/home/doc-pavi-jilan/demo-minio	993 B
194062e7	2026-01-23 18:16:02	NotVerySecure-PC	auto-pvdv-secure	/home/doc-pavi-jilan/demo-minio	998 B
422b03b5	2026-01-24 17:18:37	NotVerySecure-PC	auto-pvdv-secure	/home/doc-pavi-jilan/demo-minio	1.865 KiB

10 snapshots

Les données brutes sont correctement chiffrée par Restic sur la machine client, avant l'envoi sur le serveur MinIO :



Tout ces « blobs » de données n'ont aucun sens s'ils ne sont pas liés ensemble par les métadonnées que crée Restic dans les autres dossiers du *repo*.

Les *snapshots* sont donc chiffrés avant de passer en HTTPS par Tailscale pour finir dans la partition du disque dur MinIO /secure-backups qui est chiffrée via LUKS.

Les « data in use » sont protégée (sur le serveur MinIO) car chiffrée avant d'y arriver :

```
minio-admin@VerySecureBackup-DB:/mnt/minio-usb/secure-backups$ ls
lost+found  pvdv-backup-bucket  pvdv-backup-demo
minio-admin@VerySecureBackup-DB:/mnt/minio-usb/secure-backups$ cd pvdv-backup-bucket/
minio-admin@VerySecureBackup-DB:/mnt/minio-usb/secure-backups/pvdv-backup-bucket$ ls
config data index keys locks snapshots
minio-admin@VerySecureBackup-DB:/mnt/minio-usb/secure-backups/pvdv-backup-bucket$ ls data/
1f 27 34 38 64 7e 81 83 ca cf d0 d7 da ed f0
minio-admin@VerySecureBackup-DB:/mnt/minio-usb/secure-backups/pvdv-backup-bucket$ ls data/1f/
1f2eea158bce13a7700820655e388d245a0e1e7990e2ce49610b2642a98d466f
```

Les « data in transit » sont triplement protégées car elles transitent chiffrées par Restic, dans un tunnel TLS Tailscale au sein d'un réseau très difficilement accessible.

Les « data at rest » sont protégées également, via LUKS, comme on peut le voir ici quand je monte le disque dur MinIO sur mon ordinateur personnel qui n'a pas la clé de déchiffrement du « minio_vault.img » :

```
# thruya @ pop-os in /media/thruya/a87cc7da-e5d6-4e07-87a5-27199bfd0aff [16:07:55]
$ ls
db1 db2 db3 lost+found minio_vault.img secure-backups

# thruya @ pop-os in /media/thruya/a87cc7da-e5d6-4e07-87a5-27199bfd0aff [16:07:56]
$ ls secure-backups

# thruya @ pop-os in /media/thruya/a87cc7da-e5d6-4e07-87a5-27199bfd0aff [16:08:30]
$
```

Mettons cette architecture à l'épreuve. Je crée un fichier contenant des nouvelles données importantes :

```
doc-pavi-jilan@NotVerySecure-PC:~/demo-minio $ echo "Voici les résultats finaux pour l'analyse du cancer de Madame X" > analyse-importante.txt
doc-pavi-jilan@NotVerySecure-PC:~/demo-minio $ cat analyse-importante.txt
Voici les résultats finaux pour l'analyse du cancer de Madame X
```

La sauvegarde automatique du dossier /demo-minio se lance toute les heures mais pour la démo j'en lance une directement grâce à l'utilisateur dédié (minio-pusher) et le script qui est normalement lancé via cron. Je lui donne un tag spécifique pour la reconnaître (sauvegarde-importante) :

```
minio-pusher@NotVerySecure-PC:~ $ source .restic-secrets
minio-pusher@NotVerySecure-PC:~ $ sudo backup-pvdv-secure.sh sauvegarde-importante
repository 59c6f6d9 opened (version 2, compression level auto)
using parent snapshot 422b03b5
[0:00] 100.00% 8 / 8 index files loaded

Files:          1 new,          1 changed,          6 unmodified
Dirs:           0 new,          3 changed,          9 unmodified
Added to the repository: 8.466 KiB (2.205 KiB stored)

processed 8 files, 2.652 KiB in 0:01
snapshot c336f89b saved
```

Je vérifie qu'elle a correctement eu lieu, on constate qu'il y a déjà une certaine profondeur de *backup* et que « sauvegarde-imporante » est bien là :

```
minio-pusher@NotVerySecure-PC:~ $ restic snapshots
```

ID	Time	Host	Tags	Paths	Size
e1c903de	2026-01-20 19:08:23	NotVerySecure-PC	demo-pvdrv-secure	/home/doc-pavi-jilan/demo-minio	341 B
d56ce54b	2026-01-23 14:17:03	NotVerySecure-PC	demo-pvdrv-secure	/home/doc-pavi-jilan/demo-minio	0 B
e11fb3e2	2026-01-23 14:23:47	NotVerySecure-PC	demo-pvdrv-secure	/home/doc-pavi-jilan/demo-minio	0 B
54644ba6	2026-01-23 14:32:38	NotVerySecure-PC	demo-pvdrv-secure	/home/doc-pavi-jilan/demo-minio	651 B
e0c4a9a1	2026-01-23 14:52:54	NotVerySecure-PC	test-tag	/home/doc-pavi-jilan/demo-minio	651 B
5d54ff4f	2026-01-23 18:13:02	NotVerySecure-PC	auto-pvdrv-secure	/home/doc-pavi-jilan/demo-minio	993 B
8a77f3aa	2026-01-23 18:14:01	NotVerySecure-PC	auto-pvdrv-secure	/home/doc-pavi-jilan/demo-minio	993 B
6e18cba8	2026-01-23 18:15:01	NotVerySecure-PC	auto-pvdrv-secure	/home/doc-pavi-jilan/demo-minio	993 B
194062e7	2026-01-23 18:16:02	NotVerySecure-PC	auto-pvdrv-secure	/home/doc-pavi-jilan/demo-minio	998 B
422b03b5	2026-01-24 17:18:37	NotVerySecure-PC	auto-pvdrv-secure	/home/doc-pavi-jilan/demo-minio	1.865 KiB
c336f89b	2026-01-25 16:35:47	NotVerySecure-PC	sauvegarde-imporante	/home/doc-pavi-jilan/demo-minio	2.652 KiB

11 snapshots

Entre-temps, des vilains *hackers* ont chiffré le disque de doc-pavi-jilan :

```
doc-pavi-jilan@NotVerySecure-PC:~ $ ls | grep demo-minio
demo-minio.locked.enc
```

Pas de soucis, je peux restaurer le dernier état très facilement depuis le compte abilité (sysadmin) :

```
sysadmin@NotVerySecure-PC:~ $ restic restore latest --target .
repository 59c6f6d9 opened (version 2, compression level auto)
[0:00] 100.00% 9 / 9 index files loaded
restoring snapshot c336f89b of [/home/doc-pavi-jilan/demo-minio] at 2026-01-25 16:35:47.423616483 +0100 CET by root@NotVerySecure-PC to .
Summary: Restored 20 files/dirs (2.652 KiB) in 0:00
sysadmin@NotVerySecure-PC:~ $ ls
home script-firewall-final.sh
sysadmin@NotVerySecure-PC:~ $ cd /home/doc-pavi-jilan/demo-minio/
sysadmin@NotVerySecure-PC:~/home/doc-pavi-jilan/demo-minio $ ls
analyse-importante.txt Desktop Downloads important-files Pictures Templates time-tracker.txt
contacts.txt Documents emails.txt Music Public test.txt Videos
sysadmin@NotVerySecure-PC:~/home/doc-pavi-jilan/demo-minio $ cat analyse-importante.txt
Voici les résultats finaux pour l'analyse du cancer de Madame X
```

Ici j'ai utilisé la fonction « latest » de Restic qui va prendre le dernier *snapshot*, mais si je veux, je peux mentionner un *snapshot* spécifique pour retourner à un état ou une date désiré(e). Je restaure ainsi une ancienne sauvegarde où le fichier « analyse-importante.txt » n'avait pas encore été créé :

```
sysadmin@NotVerySecure-PC:~ $ mkdir ancien-backup
sysadmin@NotVerySecure-PC:~ $ restic restore 422b03b5 --target /home/sysadmin/ancien-backup
repository 59c6f6d9 opened (version 2, compression level auto)
[0:00] 100.00% 9 / 9 index files loaded
restoring snapshot 422b03b5 of [/home/doc-pavi-jilan/demo-minio] at 2026-01-24 17:18:37.077838078 +0100 CET by root@NotVerySecure-PC to /home/sysadmin/ancien-backup
Summary: Restored 19 files/dirs (1.865 KiB) in 0:00
sysadmin@NotVerySecure-PC:~ $ cd /home/sysadmin/ancien-backup/home/doc-pavi-jilan/demo-minio/
sysadmin@NotVerySecure-PC:~/ancien-backup/home/doc-pavi-jilan/demo-minio $ ls
contacts.txt Desktop Documents Downloads emails.txt important-files Music Pictures Public Templates test.txt time-tracker.txt Videos
```

Si j'essaie de restaurer une sauvegarde depuis minio-pusher, je ne pourrai pas car ce dernier à une politique sur le serveur MinIO qui l'empêche de lire */data* dans le *repo* Restic :

```
minio-pusher@NotVerySecure-PC:~ $ restic restore latest --target .
repository 59c6f6d9 opened (version 2, compression level auto)
[0:00] 100.00% 9 / 9 index files loaded
restoring snapshot c336f89b of [/home/doc-pavi-jilan/demo-minio] at 2026-01-25 16:35:47.423616483 +0100 CET by root@NotVerySecure-PC to .
Load(<data/ea902f9638>, 0, 0) failed: Access Denied.
Load(<data/ea902f9638>, 0, 0) failed: Access Denied.
ignoring error for /: ReadFull(<data/ea902f9638>): Access Denied.
Load(<data/ea902f9638>, 0, 0) failed: Access Denied.
Load(<data/ea902f9638>, 0, 0) failed: Access Denied.
ignoring error for /: ReadFull(<data/ea902f9638>): Access Denied.
Summary: Restored 0 files/dirs (0 B) in 0:00
Fatal: There were 2 errors
```


Si j'essaye de supprimer des sauvegardes depuis l'ordinateur de doc-pavi-jilan, que ce soit avec minio-pusher ou sysadmin, je serai bloqué :

```
minio-pusher@NotVerySecure-PC:~ $ restic forget latest --prune
repository 59c6f6d9 opened (version 2, compression level auto)
Remove(<snapshot/c336f89bce>) failed: client.RemoveObject: Access Denied.
unable to remove snapshot/c336f89bce791466499040c89120ab779f3803bd1cb1b1715678a38ae57311a from the repository
[0:00] 0.00% 0 / 1 files deleted
1 snapshots have been removed, running prune
loading indexes...
[0:00] 100.00% 9 / 9 index files loaded
loading all snapshots...
finding data that is still in use for 10 snapshots
Load(<data/ca0cbf9a87>, 0, 0) failed: Access Denied.
Load(<data/38098e7276>, 0, 0) failed: Access Denied.
Load(<data/1f2eea158b>, 0, 0) failed: Access Denied.
Load(<data/ca0cbf9a87>, 0, 0) failed: Access Denied.
[0:00] 70.00% 7 / 10 snapshots
Access Denied.
```

```
sysadmin@NotVerySecure-PC:~ $ restic forget latest --prune
repository 59c6f6d9 opened (version 2, compression level auto)
Remove(<snapshot/c336f89bce>) failed: client.RemoveObject: Access Denied.
unable to remove snapshot/c336f89bce791466499040c89120ab779f3803bd1cb1b1715678a38ae57311a from the repository
[0:00] 0.00% 0 / 1 files deleted
1 snapshots have been removed, running prune
loading indexes...
[0:00] 100.00% 9 / 9 index files loaded
loading all snapshots...
finding data that is still in use for 10 snapshots
[0:00] 100.00% 10 / 10 snapshots
searching used packs...
collecting packs for deletion and repacking
[0:00] 100.00% 13 / 13 packs processed

to repack:          32 blobs / 11.135 KiB
this removes:       0 blobs / 0 B
to delete:          6 blobs / 1.965 KiB
total prune:        6 blobs / 1.965 KiB
remaining:          32 blobs / 9.467 KiB
unused size after prune: 0 B (0.00% of remaining size)

repacking packs
[0:00] 100.00% 11 / 11 packs repacked
rebuilding index
[0:00] 100.00% 10 / 10 indexes processed
Remove(<index/57a0366d30>) failed: client.RemoveObject: Access Denied.
[0:00] 0.00% 0 / 10 old indexes deleted
Fatal: client.RemoveObject: Access Denied.
```

Mais si j'essaye de supprimer la *latest* sauvegarde Restic depuis le compte admin du serveur MinIO (*worst case scenario*) il se passe quelque chose d'intéressant, qui nous permet de mieux comprendre le fonctionnement « magique » des objets. Prenons d'abord note du numéro de référence (ID) du dernier *snapshot* :

```
sysadmin@NotVerySecure-PC:~ $ restic snapshots
repository 59c6f6d9 opened (version 2, compression level auto)
```

ID	Time	Host	Tags	Paths	Size
e1c903de	2026-01-20 19:08:23	NotVerySecure-PC	demo-pvdv-secure	/home/doc-pavi-jilan/demo-minio	341 B
d56ce54b	2026-01-23 14:17:03	NotVerySecure-PC	demo-pvdv-secure	/home/doc-pavi-jilan/demo-minio	0 B
e11fb3e2	2026-01-23 14:23:47	NotVerySecure-PC	demo-pvdv-secure	/home/doc-pavi-jilan/demo-minio	0 B
54644ba6	2026-01-23 14:32:38	NotVerySecure-PC	demo-pvdv-secure	/home/doc-pavi-jilan/demo-minio	651 B
e0c4a9a1	2026-01-23 14:52:54	NotVerySecure-PC	test-tag	/home/doc-pavi-jilan/demo-minio	651 B
5d54ff4f	2026-01-23 18:13:02	NotVerySecure-PC	auto-pvdv-secure	/home/doc-pavi-jilan/demo-minio	993 B
8a77f3aa	2026-01-23 18:14:01	NotVerySecure-PC	auto-pvdv-secure	/home/doc-pavi-jilan/demo-minio	993 B
6e18cba8	2026-01-23 18:15:01	NotVerySecure-PC	auto-pvdv-secure	/home/doc-pavi-jilan/demo-minio	993 B
194062e7	2026-01-23 18:16:02	NotVerySecure-PC	auto-pvdv-secure	/home/doc-pavi-jilan/demo-minio	998 B
422b03b5	2026-01-24 17:18:37	NotVerySecure-PC	auto-pvdv-secure	/home/doc-pavi-jilan/demo-minio	1.865 KiB
c336f89b	2026-01-25 16:35:47	NotVerySecure-PC	sauvegarde-importante	/home/doc-pavi-jilan/demo-minio	2.652 KiB

Ensuite, en tant qu'admin sur le serveur MinIO, on vérifie sur le disque dur que ce snapshot est bien présent :

```
minio-admin@VerySecureBackup-DB:/mnt/minio-usb/secure-backups/pvdv-backup-bucket/snapshots$ ls
194062e72b0da439fa01d1963b05dc1ffd08297295ae26cd401b3db29f1e7ede c336f89bced791466499040c89120ab779f3803bd1cb1b1715678a38ae57311a
422b03b55af05ff13b2ddec4b33fd798962d9596990ea2928bf2eddf324efc6d d56ce54b4c1cfaebd33019494e174ccb6081d21d569eca8a2cd857e8c4c008b5
```

Avec les droits d'administrateurs qui sont confiés à minio-admin (car il doit pouvoir nettoyer la base de données grâce à un script de nettoyage) il est possible de supprimer entièrement une sauvegarde du *repo* Restic. Pourtant, on voit bien juste après que quand je liste le contenu du disque, le *snapshot* est toujours là. C'est normal, c'est le comportement désiré du *bucket* en mode COMPLIANCE 30DAYS. Même dans le cas d'une mauvaise manipulation par l'utilisateur le plus privilégié, les données restent écrites sur disque.

```
minio-admin@VerySecureBackup-DB:/mnt/minio-usb/secure-backups/pvdv-backup-bucket/snapshots$ restic forget latest --prune
repository 59c6f6d9 opened (version 2, compression level auto)
created new cache in /home/minio-admin/.cache/restic
[0:00] 100.00% 1 / 1 files deleted
1 snapshots have been removed, running prune
loading indexes...
[0:00] 100.00% 11 / 11 index files loaded
loading all snapshots...
finding data that is still in use for 10 snapshots
[0:00] 100.00% 10 / 10 snapshots
searching used packs...
collecting packs for deletion and repacking
[0:00] 100.00% 15 / 15 packs processed

to repack:      32 blobs / 10.818 KiB
this removes:  27 blobs / 7.731 KiB
to delete:     11 blobs / 3.700 KiB
total prune:   38 blobs / 11.432 KiB
remaining:     32 blobs / 9.467 KiB
unused size after prune: 0 B (0.00% of remaining size)

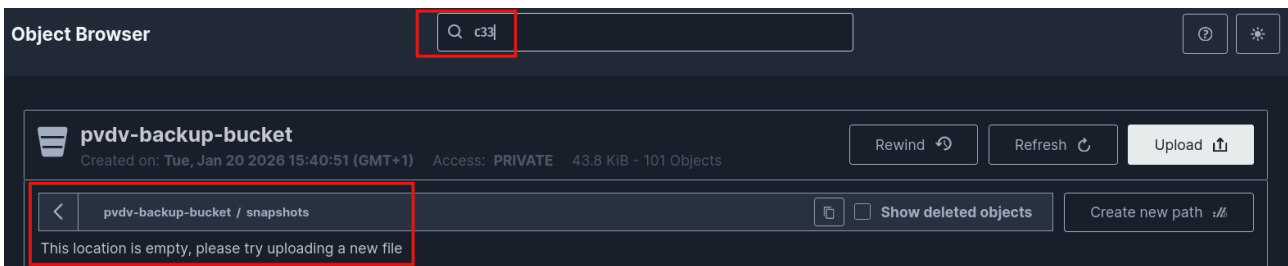
repacking packs
[0:00] 100.00% 2 / 2 packs repacked
rebuilding index
[0:00] 100.00% 12 / 12 indexes processed
[0:00] 100.00% 12 / 12 old indexes deleted
removing 6 old packs
[0:00] 100.00% 6 / 6 files deleted
done
minio-admin@VerySecureBackup-DB:/mnt/minio-usb/secure-backups/pvdv-backup-bucket/snapshots$ ls
194062e72b0da439fa01d1963b05dc1ffd08297295ae26cd401b3db29f1e7ede c336f89bced791466499040c89120ab779f3803bd1cb1b1715678a38ae57311a
```

Pourtant, si j'utilise la fonction de récupération de Restic, je n'obtiens rien. En toute logique MinIO marqué les fichiers du *snapshot latest* comme supprimés avec un marqueur DEL. Ainsi, Restic est « aveugle » et il faut procéder autrement pour récupérer notre sauvegarde.

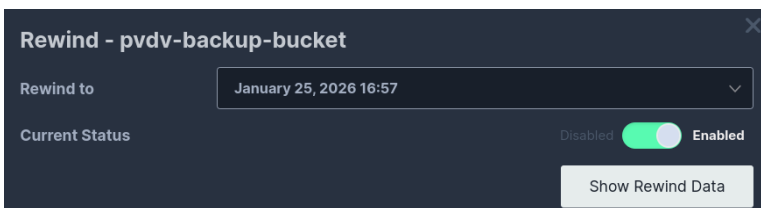
```
minio-admin@VerySecureBackup-DB:/mnt/minio-usb/secure-backups/pvdv-backup-bucket$ restic recover
repository 59c6f6d9 opened (version 2, compression level auto)
ensuring index is complete
loading indexes...
getting pack files to read...
rebuilding index
[0:00] 100.00% 1 / 1 indexes processed
[0:00] 100.00% 1 / 1 old indexes deleted
load index files
[0:00] 100.00% 1 / 1 index files loaded
load 25 trees
[0:00] 100.00% 25 / 25 trees loaded
load snapshots
done

found 0 unreferenced roots
no snapshot to write.
```


Preuve que MinIO les a marqués comme supprimé, du point de vue du *bucket* on ne voit plus rien :



Sauf si on utilise la fonction « *rewind* » qui permet littéralement de remonter dans le temps des écritures et demande de suppression du *bucket*. Je vois la création du *snapshot* ainsi que sa version marquée « *deleted* ».



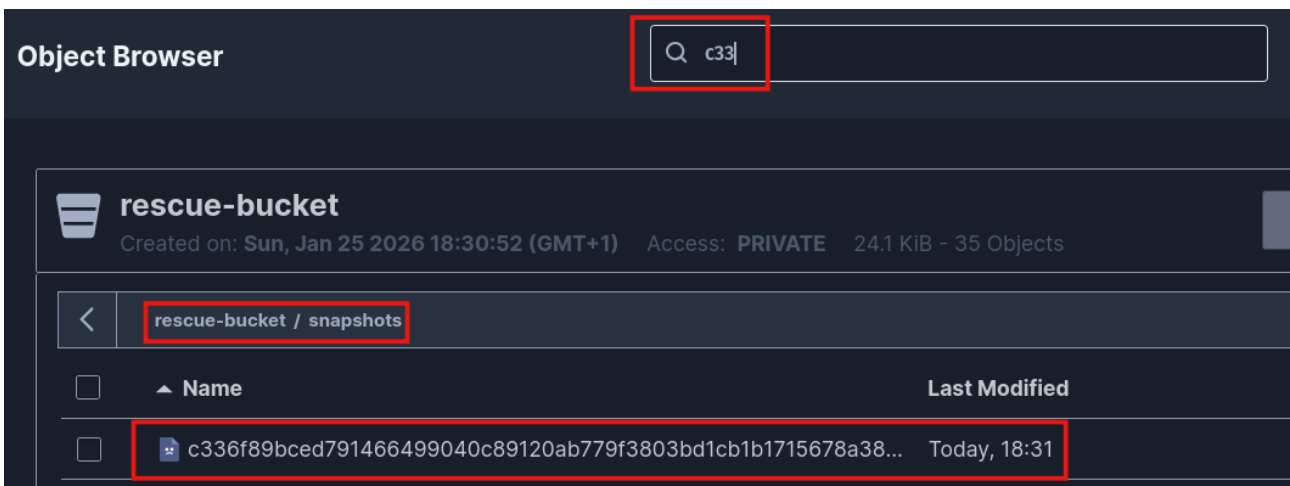
On peut également visualiser cela dans le terminal:

```
minio-admin@VerySecureBackup-DB: /mnt/minio-usb/secure-backups/pvdv-backup-bucket$ mc ls --versions --recursive pvdv_minio/pvdv-backup-bucket/snapshots/
[2026-01-23 18:16:03 CET] 448B STANDARD e7f7609f-ee8d-4066-96f9-a621966f2675 v1 PUT 194062e72b0da439fa01d1963b05dc1ffd08297295ae26cd401b3db29f1e7ede
[2026-01-24 17:18:38 CET] 447B STANDARD 137d54dc-84d1-4936-a20b-5825d4c1b66e v1 PUT 422b03b55af05ff13b2ddec4b33fd798962d9596990ea2928bf2eddf324efc6d
[2026-01-23 14:32:39 CET] 443B STANDARD ea07f3c2-a19e-40ee-b06e-fec7252a0ab9 v1 PUT 54644ba6bd9cf30f7828db772af75e27a471bbe7660b90c3f577d845c2a2b519
[2026-01-23 18:13:03 CET] 450B STANDARD 0e6cb670-70ab-49d8-8adb-c3d237a8806a v1 PUT 5d54ff4fc46709846944ec950fddadfbef4fb402fb94748e0a1cc397dd620d2a0
[2026-01-23 18:15:02 CET] 439B STANDARD 6046e52b-50fa-4b9f-a046-b3d161a8f478 v1 PUT 6e18cba8b9add16b8da433cbc17f8cb817429d78c444c0d34be1cdcaca931880
[2026-01-23 18:14:02 CET] 440B STANDARD 37415688-2818-4ed2-8e17-91b05048a801 v1 PUT 8a77f3aa5518f6278d7ac31a3a463d1ed1b84eeeb9466738ea9738395b8fed78
[2026-01-20 19:06:41 CET] 0B STANDARD a9f14828-ada2-4f91-ae27-42890677f003 v2 DEL a094e2076ae8364aec4e7d214cfafaec55a4885db74016af851aa5a90b3b3e40
[2026-01-20 18:39:33 CET] 395B STANDARD 8081620c-a9f2-4dc7-b081-fdc5759e7850 v1 PUT a094e2076ae8364aec4e7d214cfafaec55a4885db74016af851aa5a90b3b3e40
[2026-01-25 17:47:19 CET] 0B STANDARD a1c912d8-b9b8-4554-ad27-129e7201834f v2 DEL c336f89bcd791466499040c89120ab779f3803bd1cb1b1715678a38ae57311a
[2026-01-25 16:35:48 CET] 452B STANDARD d61bc78b-1720-4e29-ae95-000241ba809c v1 PUT c336f89bcd791466499040c89120ab779f3803bd1cb1b1715678a38ae57311a
[2026-01-23 14:17:04 CET] 452B STANDARD 9bbd2417-b97c-478d-be51-dcafe57fcd7b v1 PUT d56ce54b4c1cfaebd33019494e174ccb6081d21d569eca8a2cd857e8c4c008b5
[2026-01-20 18:28:42 CET] 0B STANDARD ce378e4d-b35d-456b-961b-3f2202ce65f4 v2 DEL daa0f023d456f79551db77cc868f9d3f3d7f02a589af40c9fa461ada90dd587a
[2026-01-20 17:37:05 CET] 398B STANDARD e93790d2-93be-4a1b-a02c-80ac36595527 v1 PUT daa0f023d456f79551db77cc868f9d3f3d7f02a589af40c9fa461ada90dd587a
[2026-01-23 14:52:55 CET] 433B STANDARD f4f3b3a7-7664-4d4c-910a-ea359cf3123b v1 PUT e0c4a9a1d1dfd82030477adaff89385cd112fd25e43dfdf16761299557344ebf
[2026-01-23 14:23:48 CET] 445B STANDARD 1adf500e-5916-456a-a1ba-d766cc63b6be v1 PUT e11fb3e2e3de2d9f93ec0b942e837e0c73f9a9feca36a036c158d19b068a298
[2026-01-20 19:08:24 CET] 397B STANDARD ef59f015-a18f-4990-8559-deb178387d31 v1 PUT e1c903deedd599befc1c604d88b57b3c098fdddae07bfcab05321ffa756c5c94
```

Sachant que le pvdv-backup-bucket est en mode COMPLIANCE, il serait mal avisé d'essayer de réparer directement dans celui-ci. Pour sauver la mise, je crée un rescue-bucket sans restrictions dans lequel je vais copier tout le contenu de pvdv-backup-bucket dans son état d'il y a 1h (**--rewind 1h**) :

```
minio-admin@VerySecureBackup-DB:~$ mc mb pvdv_minio/rescue-bucket
Bucket created successfully 'pvdv_minio/rescue-bucket'.
minio-admin@VerySecureBackup-DB:~$ mc cp --recursive --rewind 1h pvdv_minio/pvdv-backup-bucket/ pvdv_minio/rescue-bucket/
...ae07bfcab05321ffa756c5c94: 24.12 KiB / 24.12 KiB | 41.95 KiB/s 0s
```

La commande précédente a fonctionné, je retrouve bien tout le repo Restic dans rescue-bucket et je vérifie en recherchant le *latest snapshot* :



Enfin, pour restaurer les données, il suffit de modifier .restic-secrets pour mentionner le chemin vers le nouveau rescue-bucket. Ainsi, on retrouve notre *snapshot* qui a été malencontreusement supprimé :

```
GNU nano 8.4 .restic-rescue-secrets *
export AWS_ACCESS_KEY_ID=
export AWS_SECRET_ACCESS_KEY=
export RESTIC_REPOSITORY="s3:http://100.123.86.41:9000/rescue-bucket"
export RESTIC_PASSWORD=
```

```
minio-admin@VerySecureBackup-DB:~$ cp .restic-maintenance-secrets .restic-rescue-secrets
minio-admin@VerySecureBackup-DB:~$ nano .restic-rescue-secrets
minio-admin@VerySecureBackup-DB:~$ source .restic-rescue-secrets
minio-admin@VerySecureBackup-DB:~$ restic snapshots
repository 59c6f6d9 opened (version 2, compression level auto)
ID          Time                Host                Tags                Paths                Size
-----
e1c903de    2026-01-20 19:08:23  NotVerySecure-PC    demo-pvdv-secure    /home/doc-pavi-jilan/demo-minio  341 B
d56ce54b    2026-01-23 14:17:03  NotVerySecure-PC    demo-pvdv-secure    /home/doc-pavi-jilan/demo-minio    0 B
e11fb3e2    2026-01-23 14:23:47  NotVerySecure-PC    demo-pvdv-secure    /home/doc-pavi-jilan/demo-minio    0 B
54644ba6    2026-01-23 14:32:38  NotVerySecure-PC    demo-pvdv-secure    /home/doc-pavi-jilan/demo-minio    651 B
e0c4a9a1    2026-01-23 14:52:54  NotVerySecure-PC    test-tag            /home/doc-pavi-jilan/demo-minio    651 B
5d54ff4f    2026-01-23 18:13:02  NotVerySecure-PC    auto-pvdv-secure    /home/doc-pavi-jilan/demo-minio    993 B
8a77f3aa    2026-01-23 18:14:01  NotVerySecure-PC    auto-pvdv-secure    /home/doc-pavi-jilan/demo-minio    993 B
6e18cba8    2026-01-23 18:15:01  NotVerySecure-PC    auto-pvdv-secure    /home/doc-pavi-jilan/demo-minio    993 B
194062e7    2026-01-23 18:16:02  NotVerySecure-PC    auto-pvdv-secure    /home/doc-pavi-jilan/demo-minio    998 B
422b03b5    2026-01-24 17:18:37  NotVerySecure-PC    auto-pvdv-secure    /home/doc-pavi-jilan/demo-minio    1.865 KiB
c336f89b    2026-01-25 16:35:47  NotVerySecure-PC    sauvegarde-importante /home/doc-pavi-jilan/demo-minio    2.652 KiB
-----
11 snapshots
```

En travaillant avec Gemini, j'ai trouvé une autre méthode très efficace pour revenir à un état initial avec un script en mode interactif dans le terminal, et **directement dans le bon bucket même si il est en mode COMPLIANCE** :

```
minio-admin@VerySecureBackup-DB:~$ nano bucket-recovery.sh
minio-admin@VerySecureBackup-DB:~$ chmod +x bucket-recovery.sh
minio-admin@VerySecureBackup-DB:~$ ./bucket-recovery.sh
=== ASSISTANT DE RESTAURATION MINIO ===
Ce script va scanner un bucket pour trouver les fichiers supprimés
(DeleteMarkers) à une date spécifique et les restaurer.

Entrez le chemin complet du bucket (ex: myminio/mon-bucket) : pvdv_minio/pvdv-backup-bucket
Entrez la date à cibler (Format YYYY-MM-DD, ex: 2026-01-25) : 2026-01-25

Analyse en cours pour le bucket 'pvdv_minio/pvdv-backup-bucket' à la date du '2026-01-25'...
Scan terminé !
Nombre de fichiers prêts à être restaurés (DeleteMarkers) : 34

Voulez-vous procéder à la suppression de ces marqueurs (Restauration) ? (y/n) y

Restauration en cours...
Traitement : 34 / 34
✓Opération terminée avec succès !
Les fichiers supprimés le 2026-01-25 sont de nouveau visibles.
Vous pouvez relancer 'restic snapshots' pour vérifier.
```

La méthode consiste à chercher, dans le fichier JSON qui décrit le *bucket* visé, les fameuses versions de fichiers avec le marqueurs « DEL », et ce pour une période encodée en interactif. Ensuite, grâce à l'outil de manipulation de fichier **jq**, on crée une liste qui répertorie ces fichiers avec leur identifiants de version. Cette liste est finalement passée dans une boucle qui supprime les fichiers marqués « DEL » avec :

```
mc rm --version-id "$vid" "$TARGET_BUCKET/$key"
```

Une [autre méthode très similaire](#) existe avec un autre outil que **mc** qui permet aussi la gestion de serveurs S3.

4.5. Limites de la contre-mesure

Dans l'état où elle se trouve, cette solution est plutôt robuste d'un point de vue Protect. Cependant dû à un élément manquant majeur, que je n'ai pas eu le temps de simuler, la triade CIA n'est pas entièrement respectée. La confidentialité des données est mise en place et l'intégrité des données est conservée (si tout va bien), mais la disponibilité des données n'est pas assurée car je n'ai pas configuré de redondance des données. En effet, si la salle des serveurs de PVDV prenait feu, qu'elle était sabotée, que tous les disques durs lâchaient inopinément, se feraient voler, ou si un administrateur faisait un `sudo rm -rf` sur les disques du serveur MinIO, l'entièreté des données locales seraient perdues. L'intégrité et la disponibilité des données seraient alors réduites à néant. Il convient donc de mettre en place de la redondance sur un serveur « cold storage » compatible S3 et géographiquement distant de la clinique. Il serait possible de créer son propre *cold storage* mais il existe des solutions très peu coûteuse et reconnues. Le bas prix naît de la fonction même de ces types de serveurs : le but n'est pas d'y accéder souvent ni avec des débits d'*uploading* et *downloading* très élevés. Une solution européenne (française) et peu coûteuse (2 euros / TB) : [OVH Cold Archive Storage Service](#). On y active les fonction de *versioning* et d'*object locking* et il suffit alors de pousser les données vers ce serveur distant avec une fréquence pertinente (par exemple une fois par jour). Ainsi en cas de catastrophe majeure, les données sont restaurables !

D'autres limitations pour cette contre-mesure pourraient être:

- Le coût et la technicité de **mise en place**. Bien que j'ai passé relativement peu de temps (si on transposait ça à un employé qualifié travaillant en régime temps plein), il fût complexe de faire en sorte que tout fonctionne correctement et de manière sécurisée. Il reste d'ailleurs encore plein de petits points d'amélioration qui devraient d'office voir le jour si cette solution était déployée dans un contexte professionnel. N'ayant pas idée de coûts que ce genre de dispositif pourrait engendrer à son installation, j'indique cela comme une potentielle limitation.
- Le coût et la technicité de **maintenance**. Le stockage de données, ça coute. Malgré la partie automatisée du processus de sauvegarde et de nettoyage, il faudra réviser régulièrement le système pour s'assurer que tout fonctionne correctement, ce qui aura un coût non-négligeable. De plus, je n'ai aucune idée des quantités de données qu'un système médical engendrerait. Peut être que le coût du stockage de toutes ces données serait également à prendre en compte comme une « limite ».
- La sécurisation en profondeur (d'un point de vue Protect et pas que) du serveur MinIO pour s'assurer que ce « dernier rempart » soit (quasi) impénétrable.
- Dû au blocage du port SSH (en temps normal, sauf lors de ma démo), le serveur MinIO ne peut pas être piloté à distance (volontairement). Il faut d'office y accéder physiquement via un port Ethernet « Management » dédié sur le rack serveur lui-même. Cela pourrait être considéré aussi comme une limitation mais je vois ça comme une sécurité supplémentaire.

4.6. Approche pour l'institutionnalisation de la contre-mesure

La contre mesure mise en place pour se prévenir d'un rançongiciel m'a fait passé par une myriade de contre-mesures différentes. Celles-ci viennent garantir certains aspects requis dans la fonction Protect pour les profils « Important » CyFun2025 comme :

- *PR.DS-01 : The confidentiality, integrity, and availability of data-at-rest are protected.*
- *PR.DS-11 : Backups of data are created, protected, maintained, and tested.*
- *PR.AA-05 : Access permissions, entitlements, and authorisations are defined in a policy, managed, enforced, and reviewed, and incorporate the principles of least privilege and separation of duties.*
- *PR.IR-01 : Networks and environments are protected from unauthorised logical access and usage.*

Mais ces contre-mesures couvrent également d'autres fonctions, comme par exemple :

- *DE.CM-09 : Computing hardware and software, runtime environments, and their data are monitored to find potentially adverse events.*
- *RC.RP-05 : The integrity of restored assets is verified, systems and services are restored, and normal operating status is confirmed.*

4.6.1. Politique

Il est possible de mélanger toutes ces contre-mesures dans une politique pour PVDV :

Cette politique vise à garantir la disponibilité et l'intégrité des données des patients face aux menaces de type rançongiciel et à assurer une reprise d'activité rapide en cas d'incident, conformément au cadre CyFun 2025 (PR.DS-11, PR.DS-01).

1. **Immutabilité des sauvegardes** : Pour contrer les rançongiciels, toutes les sauvegardes critiques doivent être stockées sur un support immuable (Object Locking) empêchant toute modification ou suppression avant une période de rétention définie.
2. **Règle de redondance** : Les données doivent exister en au moins deux exemplaires, dont un situé sur un dispositif géographiquement séparé et sécurisé.
3. **Chiffrement de bout en bout** : Les données doivent être chiffrées lors de leur transfert et lorsqu'elles sont stockées (au repos) sur le serveur de sauvegarde.
4. **Moindre privilège** : Les processus automatisés de sauvegarde doivent utiliser des comptes de service dédiés, disposant uniquement des droits d'écriture, sans capacité d'écrasement ou de suppression.
5. **Segmentation réseau** : Le flux de sauvegarde doit transiter par un canal réseau sécurisé et segmenté, inaccessible au trafic utilisateur standard.

4.6.2. Procédures

De cette politique découlent une pléthore de procédures envisageable. Je me contenterai d'en lister quelques unes en guise d'exemple :

- Une exécution automatisée des sauvegardes doit avoir lieu avec un outil comme **cron**. La fréquence sera pertinente par rapport aux horaires de travail de la clinique. Par exemple, toutes les heures de 8h à 20h. Le script d'automatisation doit renvoyer une alerte si il ne parvient pas à faire ses sauvegardes.
- Le serveur de sauvegarde doit être maintenu périodiquement. L'accès physique au serveur est strictement interdit sauf pour cette maintenance périodique. L'accès par SSH doit être bloqué et le montage du disque dur chiffré doit avoir lieu correctement. Cette maintenance doit être documentée.
- Des tests de restauration doivent avoir lieu mensuellement (tous les premiers lundis du mois). Elle doit prouver la restauration d'une sauvegarde datant de 10 jours et de la sauvegarde la plus récente. Le succès de l'opération doit être documenté.
- Le plan d'urgence doit être suivi en cas d'attaque par rançongiciel sur la clinique PVDV. Ce plan pourrait contenir, par exemple, une mesure imposant le débranchement du câble réseau du serveur MinIO jusqu'à nouvel ordre.

D. Conclusions

Au terme de ce projet, l'analyse de la Polyclinique Vandervelde met en lumière une réalité incontournable du secteur de la santé : **la cybersécurité n'est pas juste un produit que l'on achète**, mais un processus continu qui doit s'adapter à la surface d'attaque.

À travers les trois étapes du scénario fictif d'attaque — de l'intrusion physique à l'attaque par rançongiciel en passant par le sabotage via périphérique HID — j'ai pu démontrer que la sécurité d'une structure, même modeste, ne repose pas sur une solution unique. Elle exige une approche **pluridisciplinaire** intégrant le concept de **défense en profondeur**. Les mesures techniques mises en place, telles que le verrouillage par *USBGuard*, la surveillance active via *Wazuh*, le renforcement de la sécurité physique ou l'immuabilité des sauvegardes avec *MinIO*, ne sont pertinentes que si elles s'inscrivent dans une **gouvernance cohérente**, dictée ici par le framework CyFun 2025.

Ce travail m'a également permis de confronter la théorie à la pratique technique. En me mettant dans la peau des employé·e·s de PVDV et en lui administrant un niveau d'assurance CyFun, j'ai été à même de comprendre les enjeux d'une telle initiative. De l'autre côté, l'implémentation réelle (scripts, configuration Docker, *hardening* des systèmes d'exploitation, etc.) révèle des défis d'ingénierie concrets. Nous avons vu que même avec des outils *open-source* et un budget limité, il est possible d'atteindre un niveau de résilience assez élevé, capable de déjouer des attaques destructrices comme les *Wipers*.

Cependant, la technologie a ses limites. Comme évoqué tout au long de ce rapport, le facteur humain reste le maillon le plus imprévisible. Aucune configuration de pare-feu ne pourra empêcher un employé contraint par la force ou dupé par ingénierie sociale d'ouvrir une porte. C'est pourquoi la fonction *Protect* ne peut se suffire à elle-même et doit impérativement s'accompagner d'une sensibilisation constante et d'une intégration fluide aux processus métiers du personnel soignant.

En définitive, la cybersécurité à la Polyclinique Vandervelde ne s'arrête pas à la remise de ce rapport. Elle devra évoluer avec les menaces. Les étapes suivantes seraient l'externalisation d'une copie des sauvegardes vers un *Cold Storage* distant pour garantir une disponibilité totale en cas de désastre, l'audit régulier des logs générés par les sondes, etc.

C'est un travail constant, et malgré tout le travail que j'ai fourni, PVDV n'est encore qu'à la ligne de départ du marathon lui permettant d'atteindre la Médaille « Important » de CyFun 2025.